



Guía de Estudio — Defensa Evaluación 3

Asignatura: ITY1101 Gestión de Datos para IA · Duoc UC · **Caso:** Telco Customer Churn **Equipo:** Benjamín Heresmann · Diego Hernández

Esta guía reúne **todo lo importante para la defensa**: la teoría del curso (módulos 3.1–3.4), la lógica de lo que construimos, cómo funciona y se conecta, el código Python explicado, y un banco de preguntas. Está escrita para **aprender y entender**, no solo para memorizar.

⚡ El proyecto en 1 minuto

Construimos un sistema completo de **predicción de abandono de clientes (churn)**: 1. Un **pipeline DataOps** (4 etapas) deja **7.043 clientes limpios** en **Supabase** (PostgreSQL). 2. Un **modelo de IA** (clasificación binaria) aprende a predecir el churn → elegimos **Regresión Logística balanceada** (**recall 79,7%**, F1 0,62, **Gini 0,69**, accuracy 74,2%). 3. Dos **microservicios** separados: **trainer** (entrena y guarda el modelo) y **predictor** (predice sin re-entrenar), comunicados solo por Supabase. 4. Un **dashboard BI** (Streamlit) muestra los resultados (KPIs, matriz de confusión, clientes en riesgo). 5. **Seguridad** auditada (RLS, privilegio mínimo) con enfoque *compliance by design* hacia la **Ley 21.719**.



Cómo se reparte la nota (¡el 70% es ORAL e individual!)

Indicador	Qué evalúa	Peso
8	Explicar y defender las MÉTRICAS del modelo (oral)	30%
9	Defender la SEGURIDAD + Ley de datos personales (oral)	30%
10	Cómo implementarían las mejoras (oral)	10%
1–7	Informe (15%) + presentación grupal (15%)	30%

Regla de oro: ambos deben dominar **TODO** — el profe pregunta al azar. Foco máximo en los **indicadores 8 (métricas) y 9 (seguridad+ley)**.

Índice

1. Fundamentos de IA supervisada y métricas del modelo
2. Arquitectura del sistema: cómo se conecta todo
3. El código Python explicado
4. Análisis de rendimiento en la nube
5. Auditoría de seguridad y Ley 21.719

- 6. Integración BI, el dashboard, y limitaciones/mejoras
- 7. Banco de preguntas y respuestas para la defensa

Fundamentos de IA supervisada y métricas del modelo

Por qué importa esta sección. Es el corazón del **indicador 8 (30% de la nota, oral e individual)**. Aquí cada uno tiene que demostrar que *entiende* el modelo que entrenó: por qué es supervisado, por qué es clasificación, cómo se evaluó y —sobre todo— **por qué elegimos Regresión Logística balanceada en vez de un modelo con "mejor accuracy"**. No basta con leer los números: hay que saber *contar la historia* detrás de ellos.

1. ¿Qué es modelar y qué es el aprendizaje supervisado?

Modelar en IA significa construir una **representación matemática que aprende a resolver un problema a partir de datos**. Un modelo no es magia: es un conjunto de reglas o relaciones que transforma *datos de entrada* (las características del cliente) en una *salida útil* (¿se va a ir o no?).

La clave está en *cómo* aprende. Existen varios **paradigmas**, pero los dos que tienen que distinguir sí o sí son:

Paradigma	Cómo aprende	Ejemplo de nuestro caso
Supervisado	Se le muestran ejemplos donde ya conocemos la respuesta correcta (datos etiquetados). Aprende corrigiéndose cada vez que se equivoca.	Tenemos 7.043 clientes históricos y de cada uno sabemos si se fue (Churn = Yes/No). El modelo aprende de esas respuestas conocidas.
No supervisado	Recibe solo datos, sin respuestas , y debe descubrir patrones o grupos por su cuenta.	Agrupar clientes parecidos <i>sin saber</i> de antemano a qué grupo pertenecen (clustering, ej. K-means).

Nuestro proyecto es **100% supervisado**: la columna **Churn** es la "respuesta correcta" que le damos al algoritmo durante el entrenamiento. Esa es justamente la condición que lo define.

💡 **Para la defensa:** "Es aprendizaje **supervisado** porque entrenamos con datos **etiquetados**: de los 7.043 clientes ya sabíamos quién había abandonado. El algoritmo aprende mirando esas respuestas conocidas y corrigiéndose; en uno *no supervisado* no le daríamos la respuesta y tendría que descubrir grupos solo."

2. Variable OBJETIVO vs. variables PREDICTORAS

Todo modelo supervisado tiene dos tipos de variables:

- **Variable objetivo (target / y):** lo que **queremos predecir**. En nuestro caso es **Churn** (abandona: Sí / No). Es la salida del modelo.

- **Variables predictoras (features / x):** la información que el modelo **usa para predecir**. Describen al cliente: antigüedad (`tenure`), tipo de contrato, método de pago, cargos mensuales, servicios contratados, etc.

Piénsenlo como una ecuación: `Churn = f(antigüedad, contrato, cargos, servicios...)` . El entrenamiento es el proceso de encontrar esa función `f` que mejor relaciona las predictoras con el objetivo.

💡 **Para la defensa:** "La **variable objetivo** es `Churn` , lo que predecimos. Las **predictoras** son las características del cliente: antigüedad, tipo de contrato, cargos mensuales... El modelo aprende cómo esas características se relacionan con la decisión de irse."

3. ¿Por qué churn es CLASIFICACIÓN BINARIA?

Un modelo supervisado puede resolver distintos **tipos de problema**. Hay que saber ubicar el nuestro:

- **Clasificación binaria** → predice entre **dos categorías**. ("¿El cliente se va? → Sí / No") ← **NUESTRO CASO**
- **Clasificación múltiple** → elige entre 3+ categorías ("Alto / Medio / Bajo").
- **Regresión** → predice un **número continuo** ("¿cuánto gastará el cliente el próximo mes?").

Churn es **binario** porque la respuesta solo tiene **dos valores posibles: abandona (1) o no abandona (0)**. En el material esto se llama también "**modelo de riesgo**": calcular la probabilidad de que ocurra un *evento no deseado* (Riesgo = Sí/No), exactamente como el "riesgo de abandono de clientes".

Si en cambio quisiéramos predecir *el monto* que gastará un cliente, eso sería **regresión** (un número), no clasificación. Ese contraste es útil mencionarlo: demuestra que entienden *por qué* el nuestro es binario y no otra cosa.

💡 **Para la defensa:** "Es **clasificación binaria** porque la variable objetivo solo tiene dos valores: se va o no se va. Es un **modelo de riesgo**: estimamos la probabilidad de un evento no deseado, el abandono. Si predijéramos *cuánto* gasta el cliente sería regresión, pero aquí solo nos interesa Sí/No."

4. División TRAIN / TEST y por qué ESTRATIFICAR

No se puede entrenar y evaluar con los mismos datos: sería como darle a un alumno el examen con las respuestas ya escritas. Por eso **partimos el dataset en dos**:

- **Entrenamiento (train, 70% ≈ 4.930 clientes):** el modelo **aprende** con estos.
- **Prueba / holdout (test, 30% = 2.113 clientes):** datos que el modelo **nunca vio**, para medir si de verdad *generaliza*.

Usamos una división **70/30 estratificada** (queda registrado así en `resumen_modelo.json`). El 70/30 equilibra dos necesidades opuestas: suficientes datos para aprender, y suficientes para evaluar de forma confiable.

¿Qué es estratificar y por qué fue obligatorio aquí? Nuestro dataset está **desbalanceado**: solo el **26,54% de los clientes abandona** (la clase "Sí" es minoría; ~3 de cada 4 clientes se quedan). Si partiéramos *al azar puro*, por mala suerte el conjunto de prueba podría quedar con muy pocos casos de churn y la evaluación no sería representativa.

Estratificar significa partir manteniendo **la misma proporción de clases en train y en test**: ~26,5% de "se va" en *ambos* lados. Así garantizamos que tanto el aprendizaje como la evaluación reflejen la realidad del problema.

💡 **Para la defensa:** "Dividimos **70/30 estratificado**: 70% para aprender, 30% (2.113 clientes) como *holdout* que el modelo nunca vio. **Estratificamos** porque solo el **26,5% abandona**; sin estratificar, el azar podría dejar el test con muy pocos casos de churn y la medición sería poco fiable. Estratificar mantiene ese 26,5% idéntico en ambos lados."

5. El flujo: DISEÑO → fit() → predict()

El entrenamiento sigue siempre la misma secuencia. Conviene tenerla clarísima porque es literalmente lo que hace nuestro microservicio **trainer**:

1. **Diseño:** decisiones *previas* — qué problema (clasificación binaria), qué algoritmo (regresión logística), qué hiperparámetros, qué métrica priorizar (recall). Todavía no se programa nada.
2. **Construcción:** se crea el objeto del modelo, p. ej. `LogisticRegression(class_weight='balanced')`. Queda listo pero **aún no ha aprendido**.
3. **Entrenamiento** — `fit(X_train, y_train)`: aquí el modelo **aprende**. Calcula internamente los **coeficientes/pesos** que mejor relacionan las predictoras con `Churn`, minimizando el error. *(En nuestra arquitectura, esto lo hace el endpoint `POST /train` del trainer, y guarda el modelo serializado en la tabla `modelo_artefacto` de Supabase.)*
4. **Evaluación** — `predict(X_test)`: el modelo ya entrenado predice sobre el **holdout** y comparamos sus respuestas con la realidad para calcular las métricas. *(Esto lo hace el predictor en `GET /metrics` y `/predict`, cargando el modelo sin re-entrenar.)*

La idea central: `fit()` = **aprender**, `predict()` = **aplicar lo aprendido**. Y son procesos separados, ejecutados por servicios distintos.

💡 **Para la defensa:** "El flujo es diseño → `fit()` → `predict()`. En `fit()` el modelo **aprende** los coeficientes con el 70% de los datos; en `predict()` los **aplica** al 30% que nunca vio. En nuestra arquitectura está separado: el **trainer** hace `fit()` y guarda el modelo en `modelo_artefacto`; el **predictor** solo carga y hace `predict()`, sin re-entrenar."

6. HIPERPARÁMETROS

Cuidado con confundir dos cosas:

- **Parámetros** (coeficientes, pesos): los **aprende el modelo solo** durante `fit()`.
- **Hiperparámetros:** configuraciones que **definimos nosotros ANTES de entrenar**; el modelo *no* los aprende. Ejemplos del material: profundidad máxima de un árbol (`max_depth`), número de árboles en Random Forest (`n_estimators=100`), tipo de regularización en regresión logística (`penalty='L2'`).

En nuestro proyecto el hiperparámetro clave fue `class_weight='balanced'` en la regresión logística. Le dice al modelo: "*pesa más los errores sobre la clase minoría (los que se van)*", justamente para compensar ese desbalance

del 26,5%. Sin él, el modelo tiende a ignorar a los que abandonan (que es lo que NO queremos).

💡 **Para la defensa:** "Los **hiperparámetros** los fijamos nosotros antes de entrenar, no los aprende el modelo. El nuestro clave fue `class_weight='balanced'` : hace que el modelo le dé más peso a la clase minoritaria —los que abandonan— para que no los ignore por el desbalance del 26,5%."

7. OVERFITTING vs. UNDERFITTING (con nuestro Random Forest como ejemplo de oro)

Estos dos conceptos son **los más probables de que pregunte el profe**, y tenemos un ejemplo perfecto en nuestros propios datos.

- **Underfitting (subajuste):** el modelo aprende **demasiado poco**, no capta los patrones. Va mal **tanto en train como en test**. (Ej.: un modelo lineal trivial para un problema complejo.)
- **Overfitting (sobreajuste):** el modelo **memoriza** los datos de entrenamiento en vez de *generalizar*. Va **excelente en train pero falla con datos nuevos**. La señal de alarma: **una brecha grande entre el rendimiento en train y en test**.

Nuestro Random Forest balanceado es un caso de libro de overfitting. Miren los números reales de `metricas_modelos.csv` :

Random Forest balanceado	Recall en TRAIN	Recall en TEST (holdout)
	100% (<code>recall_train = 1.0</code>)	63,5% (<code>recall = 0.6346</code>)

Detectó el **100% de los abandonos en los datos que ya conocía** (memorizó cada caso), pero al enfrentarse a clientes nuevos cayó al **63,5%**. Esa **brecha de ~36 puntos** entre lo que sabe "de memoria" y lo que logra "en la vida real" es la firma inconfundible del sobreajuste. Un modelo así *engaña*: parece perfecto en el laboratorio y decepciona en producción.

Comparen con la **Regresión Logística balanceada**, que elegimos: recall **80,4% en train** vs. **79,7% en test** → prácticamente **idénticos**. Esa **consistencia train/test** es la prueba de que **generaliza bien y NO sobreajusta**. Por eso un modelo con números "más modestos pero honestos" le gana a uno con un 100% ilusorio.

💡 **Para la defensa:** "El Random Forest **sobreajusta**: logra **100% de recall en entrenamiento pero solo 63,5% en el holdout** — memorizó en lugar de generalizar, y esa brecha enorme lo delata. Nuestra Logística balanceada da **80,4% en train y 79,7% en test, casi iguales**: eso prueba que generaliza. Preferimos un modelo honesto y estable a uno con un 100% que se desploma con datos nuevos."

8. MÉTRICAS A FONDO

Ahora lo medular del indicador 8: *saber leer las métricas y defender por qué priorizamos recall*.

8.1 La matriz de confusión (TP / FP / TN / FN)

Es una tabla que cruza **lo que predijo el modelo** contra **lo que pasó en realidad**. En churn, definiendo "Positivo = el cliente abandona":

	Realidad: Se fue	Realidad: Se quedó
Predijo: Se va	TP (acierto en churn)	FP (falsa alarma)
Predijo: Se queda	FN (¡se nos escapó!)	TN (acierto en quedarse)

Qué significa cada uno **en nuestro caso**:

- **TP — Verdadero Positivo**: dije "se va" y **efectivamente se fue**. ✅ Lo detectamos a tiempo → podemos retenerlo.
- **FP — Falso Positivo (error tipo I)**: dije "se va" pero **se quedaba igual**. Falsa alarma → le gastamos un descuento de retención a alguien que no lo necesitaba. *Cuesta, pero es barato.*
- **TN — Verdadero Negativo**: dije "se queda" y **se quedó**. ✅
- **FN — Falso Negativo (error tipo II)**: dije "se queda" pero **se fue**. ❌ **No hicimos nada y perdimos al cliente.** *Este es el error caro.*

Estos son los números reales de nuestra **Regresión Logística balanceada** sobre el holdout de 2.113:

	Se fue (real)	Se quedó (real)
Predijo "se va"	TP = 447	FP = 431
Predijo "se queda"	FN = 114	TN = 1.121

💡 **Para la defensa**: "La matriz cruza predicción vs. realidad. El **TP (447)** son abandonos que **detectamos**; el **FN (114)** son los que **se nos escaparon**: dijimos que se quedaban y se fueron. Para el negocio, el **FN es el error más caro**, porque significa perder un cliente sin haber intentado retenerlo."

8.2 Accuracy (exactitud) — y por qué ENGAÑA con datos desbalanceados

Accuracy = (aciertos totales) / (total) = (TP + TN) / todo. Es el % de respuestas correctas en general.

Nuestra Logística balanceada tiene **accuracy = 74,2%**. Parece "menos buena" que la **LogReg baseline (81,0%)**... pero ahí está la trampa que **tenemos que saber explicar**:

Como solo el 26,5% abandona, un modelo tramposo que dijera "**NADIE se va**" a **todos** acertaría automáticamente el **~73,5%** de las veces (porque el 73,5% efectivamente se queda) — **sin detectar ni un solo abandono**. Tendría una accuracy "decente"... y sería **completamente inútil** para el negocio.

Por eso **la accuracy ENGAÑA en problemas desbalanceados**: premia adivinar la clase mayoritaria. Una accuracy alta puede esconder que el modelo es ciego al evento que *de verdad* nos importa.

💡 **Para la defensa**: "La **accuracy engaña con desbalance**. Como el 73,5% de los clientes se queda, un modelo que dijera 'nadie se va' acertaría el 73,5% **sin detectar ni un abandono**. Por eso no nos guiamos por la accuracy del 81% del modelo base: lo que importa es cuántos *abandonos reales* detectamos."

8.3 Precision

Precision = $TP / (TP + FP)$ — De todos los que **predije** que se iban, ¿cuántos **realmente** se iban? Mide cuánto puedo *confiar* en una alarma de churn.

En la Logística balanceada: $447 / (447 + 431) = 0,509 \rightarrow 50,9\%$. Es decir, de cada 2 clientes que marcamos como "en riesgo", ~1 sí se iba. Baja, sí — porque generamos **muchas falsas alarmas (FP = 431)**. Pero eso fue una **decisión consciente**: preferimos sobre-marcar (campañas de retención de más) antes que dejar escapar clientes.

8.4 Recall (sensibilidad) — LA MÉTRICA PRIORITARIA

Recall = $TP / (TP + FN)$ — De todos los que **realmente** se iban, ¿cuántos **detecté**? Mide cuántos abandonos *no se me escaparon*.

En la Logística balanceada: $447 / (447 + 114) = 0,797 \rightarrow 79,7\%$. **Detectamos casi 8 de cada 10 abandonos reales.**

¿Por qué priorizamos recall y no precision ni accuracy? Por el **costo asimétrico de los errores**:

- Un **FP** (falsa alarma) cuesta un descuento de retención a alguien que se quedaba igual → **barato, recuperable**.
- Un **FN** (cliente que se fue sin que lo detectáramos) → **perdemos el cliente y todos sus ingresos futuros** → **caro, irreversible**.

Cuando **el FN es el error más caro, la métrica que hay que maximizar es el recall**, porque el recall es precisamente la que *castiga los FN*. El propio material de la actividad lo dice explícito: "recall, porque interesa detectar el mayor número posible de abandonos". Sacrificamos algo de precision (más falsas alarmas) a cambio de **no dejar escapar clientes**, que es lo que de verdad le duele al negocio.

💡 **Para la defensa (la respuesta clave del indicador 8):** "Priorizamos **recall = 79,7%** porque en churn **el Falso Negativo es el error más caro**: un cliente que se va sin que lo detectemos es un ingreso perdido para siempre. Una falsa alarma, en cambio, solo cuesta un descuento. El recall mide justo cuántos abandonos *no se nos escapan*: detectamos casi 8 de cada 10. Por eso aceptamos bajar la precision al 51% — preferimos exceso de alarmas antes que perder clientes en silencio."

8.5 F1-score

F1 es el equilibrio (media armónica) entre precision y recall. Resume en un solo número si el modelo es bueno *detectando* (recall) Y *sin equivocarse de más* (precision). Es útil precisamente cuando hay desbalance, porque no se deja engañar como la accuracy.

Nuestra Logística balanceada tiene el **F1 más alto de los cuatro modelos: 0,6213** — por eso `resumen_modelo.json` la marca como `mejor_modelo_por_f1`. Ganó incluso a los modelos con mayor accuracy, lo que confirma que es el mejor *para este problema*.

8.6 Curva ROC-AUC y coeficiente de GINI

- **ROC-AUC (Área Bajo la Curva ROC):** mide la **capacidad de discriminación** del modelo — qué tan bien separa a los que se van de los que se quedan, *a lo largo de todos los umbrales posibles*. Va de 0,5 (azar puro, como tirar una moneda) a 1,0 (perfecto). Nuestra Logística balanceada: **AUC = 0,846**, una capacidad de separación **buena**.
- **Coefficiente de GINI:** es el AUC reescalado para que sea más intuitivo. **Gini = 2 × AUC – 1**. Va de 0 (azar) a 1 (perfecto). Verifiquenlo con nuestro número: **2 × 0,846 – 1 = 0,692** → coincide con el **Gini = 0,69** del proyecto. Es muy usado en banca/telco para reportar poder predictivo.

A diferencia de accuracy/precision/recall (que dependen del umbral de corte elegido), **ROC-AUC y Gini evalúan el modelo de forma global**, independiente del umbral. Por eso son una segunda opinión robusta: confirman que el modelo discrimina bien *en sí mismo*.

💡 **Para la defensa:** "El **AUC = 0,846** mide qué tan bien el modelo *separa* a los que se van de los que se quedan, sin depender de un umbral; 0,5 sería azar y 1,0 perfecto. El **Gini = 0,69** es ese mismo poder reescalado con la fórmula **Gini = 2·AUC – 1** ($2 \times 0,846 - 1 = 0,69$), muy usado en telco. Confirman que el modelo discrimina bien."

9. Tabla comparativa: los cuatro modelos (números reales del proyecto)

Esta es la diapositiva mental que conviene tener fija. Métricas sobre el **holdout de 2.113 clientes**:

Modelo	Accuracy	Precision	Recall	F1	AUC	Gini	Recall TRAIN vs TEST
LogReg (baseline)	81,0%	67,1%	56,0%	0,610	0,847	0,693	54,9% / 56,0%
Árbol (baseline)	79,5%	61,6%	60,6%	0,611	0,832	0,664	61,2% / 60,6%
LogReg balanceada ✅	74,2%	50,9%	79,7%	0,621	0,846	0,693	80,4% / 79,7%
RandomForest balanceado	77,1%	56,2%	63,5%	0,596	0,823	0,646	100% / 63,5% ⚠️ sobreajusta

Cómo leer esta tabla en voz alta: - Los baselines tienen **mejor accuracy** (81% / 79,5%) pero **recall pobre** (56% / 60,6%) → se les escapan demasiados abandonos. La accuracy alta es el espejismo del desbalance. - El Random Forest **sobreajusta** (100% train → 63,5% test): descartado por no generalizar. - La **LogReg balanceada** sacrifica accuracy a propósito para subir el recall a **79,7%**, tiene el **mejor F1 (0,621)**, AUC/Gini a la par del mejor, y **recall train≈test (80,4% ≈ 79,7%)** que prueba que generaliza. **Es la elección correcta para el objetivo de negocio: detectar abandonos.**

💡 **Para la defensa (cierre del indicador 8):** "Elegimos **Regresión Logística balanceada** por tres razones, no por una sola: (1) **el recall más alto, 79,7%**, que es lo que importa porque el FN es el error caro; (2) **el mejor F1, 0,621**, el equilibrio óptimo entre detectar y no errar; y (3) **no sobreajusta** —recall casi idéntico en train y test, 80,4% vs 79,7%— al revés del Random Forest, que cae de 100% a 63,5%. Tiene menos accuracy que los baselines, sí, pero esa accuracy engaña con datos desbalanceados; lo que de verdad sirve al negocio es atrapar el máximo de abandonos, y eso lo logra este modelo."

Archivos fuente de esta sección (rutas absolutas): - C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\outputs\modelo\metricas_modelos.csv - C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\outputs\modelo\resumen_modelo.json - C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\EV3\3.1.1 Diseño, entrenamiento y evaluación inicial de un modelo de IA supervisado.pdf

Arquitectura del sistema: cómo se conecta todo

Esta es la sección que te da la **foto completa**. Si en la defensa te preguntan "¿cómo funciona su sistema de principio a fin?", lo que sigue es exactamente lo que tenés que poder contar: la vida de un dato desde que es una fila en un CSV hasta que aparece pintado en el dashboard como "cliente en riesgo".

1. La idea central: un dato que viaja por 4 estaciones

Imaginá una **fábrica con una cinta transportadora**. La materia prima (el CSV crudo con 7.043 clientes) entra por un extremo, pasa por varias estaciones que la van puliendo, y sale por el otro extremo convertida en algo útil (predicciones de quién se va a ir). Nadie en la fábrica hace todo: cada estación tiene **una sola tarea** y le pasa el resultado a la siguiente.

En nuestro sistema hay **cuatro grandes "estaciones"**, y todas se hablan a través de un único punto de encuentro: **la base de datos en Supabase** (PostgreSQL 17). Esto es la clave que hay que entender: los componentes **no comparten disco, no se llaman entre sí directamente, no se pasan archivos**. Todos escriben y leen de tablas en Supabase. Supabase es el "tablón de anuncios" común.

Las cuatro estaciones, y la tabla que cada una usa:

#	Estación	Qué hace	Deja su resultado en...
1	Pipeline DataOps (4 etapas)	Limpia y valida el CSV crudo	tabla <code>clientes</code> (7.043 filas limpias)
2	TRAINER (microservicio)	Entrena el modelo de IA	tabla <code>modelo_artefacto</code> (modelo + métricas)
3	PREDICTOR (microservicio)	Puntúa a cada cliente con el modelo	tabla <code>predicciones</code> (2.914 en riesgo)
4	DASHBOARD BI (Streamlit)	Muestra métricas y clientes	(no escribe; solo lee 2 y 3)

Fijate en la cadena: la estación 1 deja `clientes`, que es **lo que come** la estación 2; la 2 deja `modelo_artefacto`, que es **lo que come** la 3; la 3 deja `predicciones`, que es **lo que come** la 4. Como una posta: cada uno entrega el testigo a través de Supabase.

 **Para la defensa:** "Nuestro sistema es una posta de cuatro etapas donde **el dato es el testigo y Supabase es la pista**. El pipeline deja clientes limpios, el trainer deja el modelo, el predictor deja las predicciones, y el dashboard las muestra. Ningún componente se comunica con otro directamente: **todos pasan por la base de datos.**"

2. La historia completa, paso a paso (el ciclo de vida del dato)

Estación 1 — El pipeline DataOps deja datos limpios en `clientes`. El CSV fuente (`data/source/telco_churn_source.csv`) entra por la etapa de **ingesta**, pasa por **limpieza** (se arregla `TotalCharges`, se normalizan booleanos, se crean features como `tenure_group`), luego por **validación** (con `pandera`; lo que no cumple las reglas se aparta a `clientes_rechazados`), y finalmente la etapa de **carga** (`carga_bd.py`) inserta los 7.043 registros válidos en la tabla `clientes` de Supabase.

Un detalle que conviene saber decir: esa carga es **idempotente con full-refresh**. En el código (`carga_bd.py`, función `cargar`) se ve que antes de insertar hace `TRUNCATE TABLE clientes` y luego el `INSERT`, **todo dentro de la misma transacción**. ¿Por qué? Para que correr el pipeline dos veces dé exactamente el mismo resultado (principio DataOps de **reproducibilidad**), y porque si el insert falla, el `ROLLBACK` deja la tabla intacta — no quedan datos a medias. La tabla de auditoría `carga_logs`, en cambio, **nunca se borra**: ahí queda el historial de cada corrida.

Estación 2 — El TRAINER entrena y guarda el modelo en `modelo_artefacto`. El microservicio trainer (`serve_modelo.py` con `ROL=trainer`) expone un único botón: `POST /train`. Cuando lo disparás, hace tres cosas (líneas 53–61 de `serve_modelo.py`): 1. `modelo.cargar_datos("supabase")` → lee los 7.043 de `clientes`. 2. `modelo.entrenar_modelo(df)` → entrena la Regresión Logística balanceada. Evalúa en un **holdout 70/30 estratificado** (2.113 clientes de prueba) para reportar métricas honestas — y de ahí salen el **recall 79,7%, F1 0,62, Gini 0,69, accuracy 74,2%** — y luego **re-ajusta con todos los datos** para que el modelo final sea el mejor posible. 3. `modelo.guardar_modelo_supabase(pipe, met)` → **serializa el modelo** (con `joblib`, codificado en `base64`) y lo guarda como una fila en la tabla `modelo_artefacto`, junto con sus métricas.

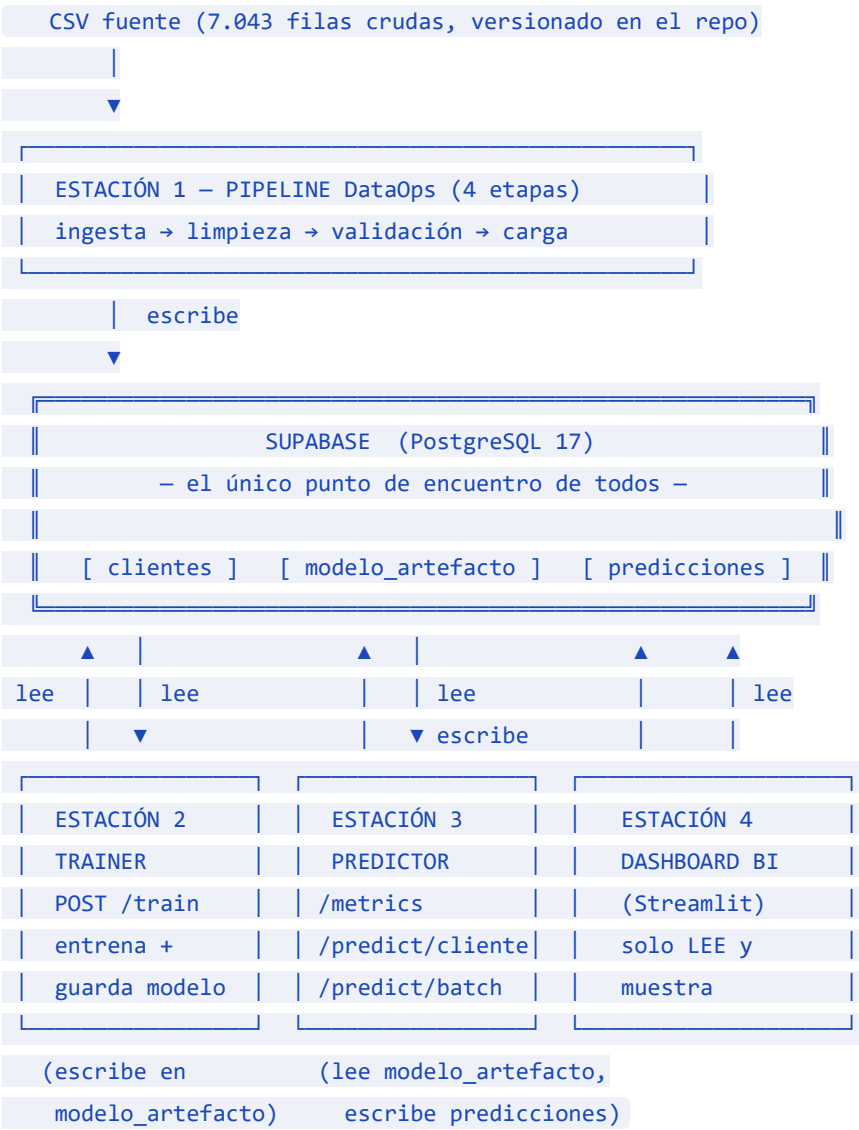
Lo importante: el modelo entrenado **no queda en un archivo en el disco del trainer**. Queda guardado **en Supabase**, como un dato más. Por eso otro servicio puede recogerlo.

Estación 3 — El PREDICTOR carga ese modelo y escribe en `predicciones`. El microservicio predictor (mismo `serve_modelo.py`, pero con `ROL=predictor`) **nunca entrena**. Cuando arranca, `cargar_modelo_supabase()` lee la fila de `modelo_artefacto`, **deserializa** el modelo y lo guarda en una caché en memoria (`_cache`) para no leerlo de la base en cada request. Luego ofrece tres servicios: - `GET /metrics` → devuelve las métricas holdout que venían guardadas con el modelo. - `GET /predict/cliente/{id}` → predice el riesgo de **un** cliente puntual. - `POST /predict/batch` → **puntúa a los 7.043** clientes de golpe y **refresca la tabla `predicciones`** (de ahí salen los **2.914 en riesgo**).

Estación 4 — El DASHBOARD lee `modelo_artefacto` y `predicciones` y los muestra. El dashboard Streamlit **no calcula nada ni entrena nada**: es una **vitrina**. Se conecta a Supabase y lee dos tablas: de `modelo_artefacto` saca las **métricas** (matriz de confusión, recall, F1...) y de `predicciones` saca la **lista de clientes** y su nivel de riesgo. Por eso, gracias al **auto-refresh**, arranca vacío y se va llenando solo a medida que disparás `/train` (aparecen las métricas) y `/predict/batch` (aparecen los clientes) — ideal para construir el resultado en vivo durante la defensa.

💡 **Para la defensa:** "El dato vive así: el pipeline lo deja **limpio** en `clientes`; el trainer aprende de esa tabla y guarda el **modelo serializado** en `modelo_artefacto`; el predictor toma ese modelo y escribe el **scoring** en `predicciones`; y el dashboard **solo lee** esas dos tablas para mostrar las métricas y los 2.914 clientes en riesgo. **El dashboard no entrena ni predice: muestra.**"

3. El diagrama (esto es lo que conviene dibujar en la pizarra)



Leélo siguiendo las flechas: **todo sube o baja a la caja del medio (Supabase)**. Ningún cuadro de abajo toca a otro cuadro de abajo. Esa es la prueba visual de que el sistema está **desacoplado**.

💡 **Para la defensa:** "Si tuviera que dibujarlo: tres servicios abajo, una base de datos en el centro, y **todas las flechas apuntan a la base, nunca de servicio a servicio**. Eso demuestra el desacople de un vistazo."

4. ¿Por qué está DESACOPLADO? (microservicios separados)

"Desacoplado" significa que **cada pieza puede vivir, fallar, actualizarse y escalar por su cuenta** sin arrastrar a las demás. Es lo opuesto a un **monolito** (un solo programa gigante que hace todo: si una parte cae, cae todo; si querés escalar una parte, tenés que duplicar el bloque entero).

Por qué nos convino partirlo así: - **Aislamiento de fallos:** si el dashboard se cae, el modelo sigue prediciendo; si Railway se reinicia, los datos en Supabase no se pierden. - **Escalar solo lo que satura:** el predictor (que se usa mucho) se puede replicar sin tocar al trainer (que se usa poco). En un monolito tendrías que clonar todo. -

Evolucionar por separado: podemos cambiar el modelo sin tocar el dashboard, y el dashboard sin tocar el pipeline, porque el contrato entre ellos **es la tabla de Supabase**, no el código. - **Coherencia con Eval 2:** ya habíamos hecho "un contenedor por capa" en el pipeline (cada etapa su propio contenedor). En Eval 3 **extendimos ese mismo patrón a la capa de IA:** trainer y predictor son dos contenedores separados.

Un detalle elegante que vale la pena mencionar: trainer y predictor son **el mismo archivo y la misma imagen Docker** (`serve_modelo.py` / `Dockerfile.modelo`). Lo único que los diferencia es la **variable de entorno** `ROL` : si `ROL=trainer` expone `/train` ; si `ROL=predictor` expone los endpoints de predicción (se ve en la línea 25 y 52 de `serve_modelo.py`). Un solo código, dos roles — exactamente el mismo truco que en Eval 2.

💡 **Para la defensa:** "Lo partimos en microservicios para **aislar fallos y escalar solo lo que se satura**. Trainer y predictor son **la misma imagen**; lo que cambia su rol es una variable de entorno `ROL` . Es el patrón 'un contenedor por capa' de Eval 2, ahora aplicado a la IA."

5. ¿Cómo se comunican? Todo vía Supabase, nunca disco compartido

Esta es la pregunta trampa más probable: "*¿Cómo le pasa el trainer el modelo al predictor?*". La respuesta correcta **no** es "por un archivo compartido". Es:

El trainer **serializa** el modelo (joblib → base64) y lo **guarda como una fila** en la tabla `modelo_artefacto` . El predictor, que es otro contenedor independiente, **lee esa fila** y **deserializa** el modelo. Nunca tocan el mismo disco.

¿Por qué esto importa? Porque trainer y predictor pueden estar en **máquinas distintas** en Railway. No tienen una carpeta en común. Si dependieran de un archivo local, romperían en cuanto se desplegaran por separado. Al usar Supabase como intermediario, **el modelo es un dato que viaja por la red**, igual que cualquier otro registro. En el código se ve clarísimo: `serve_modelo.py` importa `modelo.guardar_modelo_supabase` y `modelo.cargar_modelo_supabase` — guardar y cargar, ambos contra Supabase. La conexión, además, va **forzada con SSL** (`sslmode=require` , ver `carga_bd.py` líneas 83–87).

💡 **Para la defensa:** "El trainer guarda el modelo **serializado dentro de Supabase**, no en un archivo. El predictor lo lee desde ahí. Se comunican **a través de la base de datos**, no de disco compartido — por eso pueden vivir en servidores distintos sin romperse."

6. TRAIN vs PREDICT: entrenar una vez, predecir muchas

Esta distinción es el corazón de por qué hay **dos** servicios y no uno:

	TRAIN (entrenar)	PREDICT (predecir)
¿Quién?	trainer	predictor
¿Cada cuánto?	Una vez (offline, esporádico)	Muchas veces (online, constante)
¿Es caro?	Sí — lee toda la base, ajusta el modelo	No — solo aplica un modelo ya hecho
¿Qué deja?	El modelo en <code>modelo_artefacto</code>	Las predicciones en <code>predicciones</code>
Analogía	Estudiar para el examen	Rendir el examen con lo estudiado

La analogía del examen funciona perfecto: **estudiás una vez** (entrenar es lento y caro), pero después **respondés muchas preguntas** con lo que aprendiste (predecir es rápido y barato). Sería absurdo volver a estudiar toda la materia cada vez que te hacen una pregunta. Por eso el predictor **carga el modelo y predice SIN re-entrenar** — es justo el comentario que está en el código (`serve_modelo.py` , línea 360 del README y el docstring del módulo).

💡 **Para la defensa:** "Entrenar es como **estudiar**: lento, caro, se hace **una vez offline**. Predecir es como **rendir el examen**: rápido, se hace **muchas veces**. Por eso separamos: el trainer estudia una vez y guarda el modelo; el predictor lo usa muchas veces sin volver a estudiar."

7. ¿Qué corre LOCAL y qué corre en la NUBE?

Conviene tener clarísima esta frontera, porque demuestra que entendés *dónde* vive cada cosa:

En tu PC (LOCAL) — para desarrollar y probar: - `src/modelo.py` como **script de línea de comandos**: `python src/modelo.py --train` (entrena y guarda) o `--predict` (carga y predice). Es la versión "de banco de trabajo" del modelo, para iterar rápido antes de desplegar. - `src/benchmark.py` para medir rendimiento, y opcionalmente la API o el dashboard con `uvicorn` / `streamlit run` en `localhost`.

En la NUBE (Railway) — lo que está desplegado y vivo 24/7: - La **API FastAPI** del pipeline (`telco-api-production`). - El **microservicio trainer** (`telco-trainer-production`). - El **microservicio predictor** (`telco-predictor-production`). - El **dashboard BI** Streamlit (`telco-dashboard-production`).

Y la **base de datos (Supabase)** está en la nube siempre — es el punto fijo al que todos, local o nube, se conectan.

La gracia: el **mismo** `modelo.py` sirve de las dos formas. Local lo invocás como script; en la nube, `serve_modelo.py` lo **importa como librería** (`import modelo`) y expone sus funciones como endpoints HTTP. Un solo código, dos formas de usarlo — local para desarrollar, nube para servir.

💡 **Para la defensa:** "`modelo.py` corre **local como script** para desarrollar (`--train` / `--predict`). En la **nube**, ese mismo código lo importa `serve_modelo.py` y lo expone como microservicios en Railway. **La base (Supabase) está siempre en la nube** y es el punto al que se conecta todo, corra donde corra."

Cierre: la frase de una sola línea

Si te queda poco tiempo y querés resumir toda la arquitectura en una frase:

"Cuatro componentes desacoplados —pipeline, trainer, predictor, dashboard— que se comunican únicamente a través de tablas en Supabase: el pipeline deja los datos limpios, el trainer guarda el modelo, el predictor escribe las predicciones y el dashboard las muestra; entrenar se hace una vez offline y predecir muchas veces, y todo lo serio corre en la nube (Railway + Supabase) mientras el código se desarrolla local."

Archivos de referencia (rutas absolutas): - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\README.md` - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\serve_modelo.py` - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\carga_bd.py` - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\modelo.py`

El código Python explicado

Esta sección recorre el código fuente archivo por archivo. La idea es que cualquiera de los dos pueda abrir el Swagger o el repo y explicar, en lenguaje simple, **qué hace cada archivo, por qué se decidió así, y qué concepto de Python/sklearn hay detrás**. Todos los archivos viven en `telco-churn-pipeline/src/`.

Antes de entrar: el sistema tiene dos grandes mitades. 1. **El pipeline DataOps** (4 etapas): `ingesta.py` → `limpieza.py` → `validacion.py` → `carga_bd.py`. Toma el CSV crudo y deja 7.043 clientes limpios en Supabase. 2. **El modelo de IA**: `modelo.py` (el cerebro: entrena, evalúa, predice) y `serve_modelo.py` (la cáscara web que lo expone como microservicios).

Una regla de oro que se repite en todos los archivos: **cada etapa lee lo que dejó la anterior y escribe en su propia carpeta** (`data/raw` → `data/clean` → `data/validated` → base de datos). Están **desacopladas**: cada una se puede correr sola, y todas usan un patrón idéntico de "buscar el último archivo con timestamp" (`sorted(...)[-1]`). Eso es lo que las hace un *pipeline* y no un solo script gigante.

1. `ingesta.py` — Etapa 1: traer el dato crudo

Qué hace en una frase: es el único punto de entrada de datos al sistema. Busca el CSV fuente, lo copia a `data/raw/` con una marca de tiempo en el nombre, y registra de dónde vino.

La función clave es `ingestar()`. Lo más interesante es su **estrategia de fuentes en cascada** (un `if/elif/else`), que prueba orígenes en orden de prioridad: 1. Una ruta explícita que le pasen como parámetro. 2. Si hay variables `SUPABASE_URL` + `SUPABASE_KEY` → descarga desde Supabase Storage. 3. Si hay `SOURCE_CSV_PATH` → lee un archivo local en esa ruta. 4. **Fallback**: el dataset versionado en el repo (`data/source/telco_churn_source.csv`), que viaja dentro de la imagen Docker.

Esto es **robustez**: el pipeline funciona en local, en la nube o sin internet, sin cambiar el código, solo cambiando variables de entorno.

Detalles que el profe puede preguntar: - El **timestamp en el nombre** (`telco_churn_raw_20260628_153000.csv`): da **trazabilidad**. Nunca se pisa un archivo; cada ingesta queda registrada. La función `datetime.now().strftime("%Y%m%d_%H%M%S")` genera ese sello. - `shutil.copy2` copia el

archivo *preservando metadatos* (fecha de modificación). Se usa `copy2` y no `copy` a propósito. - `load_dotenv` lee el archivo `.env` para cargar las variables de entorno (credenciales) sin escribirlas en el código. - Al final hace `pd.read_csv(...).shape` solo para **contar filas y columnas** y dejarlas en el log (7.043 filas, 21 columnas). - El bloque `if __name__ == "__main__":` con `sys.exit(0/1)` permite correr la etapa sola desde la terminal y que devuelva un **código de salida** (0 = éxito, 1 = error), que es lo que un orquestador necesita.

💡 **Para la defensa:** "La ingesta es el único punto de entrada de datos. Tiene una cascada de fuentes (ruta explícita → Supabase Storage → ruta local → dataset del repo) que da robustez sin tocar el código, y cada archivo se guarda con timestamp para trazabilidad: nunca pisamos un dato crudo."

2. `limpieza.py` — Etapa 2: transformar y normalizar

Qué hace en una frase: convierte el CSV crudo en un dataset prolijo: arregla tipos, traduce textos a booleanos, crea una variable nueva y elimina duplicados. **No valida** (eso es trabajo de la etapa 3); solo transforma.

La función central es `limpiar()`. Sus cuatro transformaciones, en orden:

a) El fix de `TotalCharges` (la pregunta clásica del caso). En el CSV original, `TotalCharges` viene como **texto**, y los clientes nuevos (`tenure` = 0, recién registrados, aún sin facturar) tienen ahí un **string vacío** `" "`. Eso rompería cualquier cálculo numérico. La solución, en dos pasos:

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

`pd.to_numeric` convierte la columna a número, y `errors="coerce"` significa "lo que no puedas convertir, conviértelo en **NaN**" (nulo) en vez de reventar. Luego viene una decisión de negocio:

```
mask_imputable = df["TotalCharges"].isna() & (df["tenure"] == 0)
df.loc[mask_imputable, "TotalCharges"] = 0.0
```

Es decir: si el total está vacío **y** además el cliente lleva 0 meses, su total real **es 0** (todavía no ha pagado nada), así que se **imputa 0** con fundamento, no se inventa. Esto es **imputación con criterio de negocio**, no un relleno ciego.

b) Yes/No → booleano. Hay 5 columnas claramente binarias (`Partner`, `Dependents`, `PhoneService`, `PaperlessBilling`, `Churn`). Se traducen con un `.map()`:

```
df[col] = df[col].map({"Yes": True, "No": False})
```

`.map()` reemplaza cada valor según un diccionario. Solo se aplica a columnas donde "Yes/No" es inequívoco. Columnas como `OnlineSecurity` (que tienen un tercer valor, "No internet service") **se dejan como texto** a propósito, porque convertirlas a booleano perdería información.

c) `tenure_group` (feature derivada). Se crea una columna nueva agrupando la antigüedad en meses en tramos (`0-12`, `13-24`, `25-48`, `49-72`, `73+`) con la función `_categorizar_tenure`:

```
df["tenure_group"] = df["tenure"].apply(_categorizar_tenure)
```


`.apply()` ejecuta una función sobre cada fila de la columna. Esto es **feature engineering**: una variable nueva que facilita el análisis (por ejemplo, ver qué grupo de antigüedad se fuga más). Nota: esta columna sirve para análisis, pero el modelo la **descarta** (ver `modelo.py`) para no duplicar la información que ya está en `tenure`.

d) Deduplicación. `df.duplicated(subset=["customerID"])` detecta clientes repetidos por ID y se quedan con el primero (`keep="first"`).

💡 **Para la defensa:** "El caso clásico es `TotalCharges`: viene como texto con celdas vacías. Lo pasamos a número con `pd.to_numeric(errors='coerce')` y, donde queda NaN y el cliente tiene `tenure` 0, imputamos 0 porque es un cliente nuevo que aún no ha facturado: es imputación con criterio de negocio. Además creamos `tenure_group` como feature derivada para el análisis."

3. `validacion.py` — Etapa 3: el portero de calidad (`pandera`)

Qué hace en una frase: revisa fila por fila que los datos cumplan las reglas; las filas buenas van a `data/validated/`, las malas van a `data/rejected/` **con su motivo** para auditoría. Nunca deja entrar dato sucio a la base.

Hay **dos niveles de validación**, y es importante distinguirlos:

a) Validación estructural (con `pandera`). `pandera` es una librería que define un **esquema**: para cada columna, qué tipo debe tener, qué rango y qué valores permitidos. Por ejemplo, `tenure` debe ser un entero entre 0 y 72, `Contract` solo puede ser uno de tres valores, etc. (el esquema vive en `utils/schema.py`). La validación se hace así:

```
schema_clientes.validate(df, lazy=True)
```

El `lazy=True` es clave: significa **"no te detengas en el primer error, recolecta TODOS los errores de golpe"**. Si algo falla, `pandera` lanza una excepción `SchemaErrors` con una tabla (`failure_cases`) que dice exactamente qué fila, qué columna y qué regla falló. El código atrapa esa excepción, separa las filas inválidas, y a cada una le pega un `motivo_rechazo` legible (ej. `tenure=99 (in_range)`).

b) Validación semántica (reglas de negocio cruzadas). Esto es lo que `pandera` por sí solo no hace: **coherencia entre columnas**. El ejemplo del caso: si un cliente tiene `InternetService = "No"`, entonces sus servicios derivados (`OnlineSecurity`, `StreamingTV`, etc.) **tienen que decir** `"No internet service"` — no pueden decir "Yes", sería una contradicción. Eso se chequea fila por fila con `validar_reglas_semanticas` (en `utils/schema.py`).

Las filas que pasan **ambos** filtros se guardan; las rechazadas se concatenan (`pd.concat`) con su motivo y su `tipo_validacion` (estructural o semántica). Al final calcula la **tasa de validez** (% de filas que pasaron). En este dataset el 100% pasa, pero la infraestructura para rechazar y auditar está construida.

💡 **Para la defensa:** "La validación tiene dos capas. La estructural usa `pandera` con `lazy=True` para juntar todos los errores de tipo, rango y valores permitidos de una vez. La semántica chequea coherencia entre columnas: por ejemplo, si no hay internet, los servicios de internet tienen que decir 'No internet service'. Lo que falla no se borra: se guarda en `data/rejected` con su motivo, para auditoría de calidad."

4. `carga_bd.py` — Etapa 4: cargar a PostgreSQL (Supabase)

Qué hace en una frase: toma el CSV validado y lo inserta en la tabla `clientes` de Supabase dentro de una transacción, dejando registro de auditoría de la carga.

Conceptos clave:

a) SQLAlchemy + el engine cacheado. `SQLAlchemy` es la librería que conecta Python con la base de datos. La función `_build_engine()` arma el **engine** (la puerta de entrada a Postgres). Tiene dos detalles importantes: - `@lru_cache(maxsize=1)`: este decorador hace que el engine se construya **una sola vez** y se reutilice. ¿Por qué importa? Porque la API (FastAPI) recibe muchas requests, y crear un engine nuevo por cada una **agotaría el límite de conexiones** del plan gratuito de Supabase. El engine mantiene un **pool de conexiones** (`pool_size=3`) que se reciclan. - **SSL forzado:** arma la URL con `?sslmode=require`, de modo que la conexión a Supabase va **cifrada** sí o sí.

b) El mapeo de nombres. El CSV usa `customerID`, `MonthlyCharges` (estilo del dataset); la base usa `customer_id`, `monthly_charges` (snake_case, convención SQL). El diccionario `MAPEO_COLUMNAS` traduce de uno a otro con `df.rename(columns=...)`.

c) La decisión estrella: carga full-refresh idempotente (`TRUNCATE` + `INSERT`). Esto es lo que hay que saber explicar bien:

```
with engine.begin() as conn:
    conn.execute(text("TRUNCATE TABLE clientes RESTART IDENTITY CASCADE"))
    df.to_sql("clientes", conn, if_exists="append", ...)
```

- `TRUNCATE` vacía la tabla, y luego se **reinsertan** todas las filas. ¿Por qué no `INSERT` a secas? Porque así la carga es **idempotente**: corras el pipeline 1 vez o 5 veces, el resultado final es **siempre el mismo** (7.043 clientes, sin duplicados ni acumulación). Esa reproducibilidad es un principio central de **DataOps**.
- `with engine.begin()` abre una **transacción**: el `TRUNCATE` y el `INSERT` viajan **juntos**. Si el insert falla a mitad de camino, se hace **ROLLBACK** automático y la tabla queda **intacta** (no queda vacía ni a medias). Es la propiedad de **atomicidad**: "todo o nada".
- `method="multi", chunksize=500` inserta de a 500 filas por sentencia, para que sea rápido.

d) Auditoría. Dos cosas se auditan: las filas rechazadas de la etapa 3 van a la tabla `clientes_rechazados` (guardando el dato original como JSON), y cada ejecución de la carga deja un registro en `carga_logs` (cuántas filas leyó/insertó, duración, estado OK/ERROR). Importante: **el histórico de `carga_logs` NUNCA se trunca** — solo las tablas de estado.



Para la defensa: "La carga es full-refresh idempotente: `TRUNCATE` + `INSERT` dentro de una sola transacción. Idempotente significa que correrlo N veces da siempre el mismo resultado, sin duplicados, que es un principio DataOps. Y al ir en una transacción, si el insert falla hace ROLLBACK y la tabla queda intacta. El engine se cachea con `lru_cache` para no agotar las conexiones de Supabase, y la conexión va con SSL obligatorio."

5. `modelo.py` — El cerebro: entrenar, evaluar y predecir

Este es el archivo más importante para el indicador 8 (métricas del modelo). Es el **núcleo de IA**. Tiene **tres modos de ejecución**, elegidos por argumento de línea de comandos (`argparse`):

- `python src/modelo.py --eval` → entrena y **compara 4 modelos** + genera gráficos (alimenta el informe).
- `python src/modelo.py --train` → entrena el **modelo final** y lo **guarda** en Supabase.
- `python src/modelo.py --predict` → **carga** el modelo guardado y predice, **sin re-entrenar**.

Esta separación `train` / `predict` es lo que permite el patrón de **microservicios** (un contenedor entrena, otro predice). Lo orquesta `argparse` con un `mutually_exclusive_group()`, que obliga a elegir **un solo** modo a la vez.

a) `_features(df)` — qué entra al modelo. Construye la matriz **X** (las variables predictoras) quitando lo que no debe entrar: `customerID` (es un identificador, no informa), `Churn` (es la respuesta, sería trampa) y `tenure_group` (redundante con `tenure`). Además convierte los booleanos a 0/1 con `.astype(int)`, porque sklearn trabaja con números.

b) `construir_preprocesador(X)` — el `ColumnTransformer` (concepto sklearn clave). Distintos tipos de columna necesitan tratamientos distintos. Un `ColumnTransformer` aplica una transformación diferente a cada grupo de columnas, todo en un solo objeto:

```
ColumnTransformer([
    ("num", StandardScaler(), NUM_CONTINUAS),          # escala las continuas
    ("bin", "passthrough", binarias),                  # deja las binarias tal cual
    ("cat", OneHotEncoder(handle_unknown="ignore"...), categoricas), # one-hot
])
```

- `StandardScaler` sobre las 3 continuas (`tenure`, `MonthlyCharges`, `TotalCharges`): las **estandariza** (media 0, desviación 1). Esto importa **porque la Regresión Logística es sensible a la escala**: sin escalar, una variable con números grandes dominaría a una con números chicos.
- `"passthrough"` sobre las binarias: ya son 0/1, no hay que tocarlas.
- `OneHotEncoder` sobre las categóricas (ej. `Contract`, `PaymentMethod`): convierte una columna de texto con N categorías en N columnas de 0/1. Por ejemplo `Contract` se vuelve tres columnas (`Contract_Month-to-month`, `Contract_One year`, `Contract_Two year`). El modelo no entiende texto; necesita números. El `handle_unknown="ignore"` evita que reviente si en producción aparece una categoría nunca vista.

c) El `Pipeline` de sklearn (otro concepto clave). En `_modelo_produccion` se encadenan preprocesador + clasificador en **un solo objeto**:

```
Pipeline([
    ("pre", construir_preprocesador(X)),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced", random_state=42)),
])
```

La gracia del `Pipeline` es que **el preprocesamiento y el modelo viajan juntos**. Cuando llamas `.fit()`, aprende a escalar/codificar **y** a clasificar; cuando llamas `.predict()`, aplica exactamente las mismas transformaciones. Esto **evita el "data leakage"** (que info del test se cuele en el entrenamiento) y hace que

guardar el modelo guarde también su preprocesamiento: al cargarlo en producción, ya sabe transformar el dato crudo.

d) `class_weight="balanced"` (la decisión de modelado central). El dataset está **desbalanceado**: solo el 26,5% de los clientes hacen churn. Si no haces nada, el modelo aprende a decir "nadie se va" y acierta el 73% por pura pereza... pero es inútil, porque **no detecta a los que se fugan**. `class_weight="balanced"` le dice al modelo que **pondere más los errores sobre la clase minoritaria** (los que se van). El resultado: el **recall** sube a **79,7%** (detecta a 8 de cada 10 que se van), que es justo la métrica que más importa en retención.

e) `train_test_split` con `stratify` (concepto clave de evaluación).

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.30, stratify=y, random_state=42)
```

- Parte los datos en **70% entrenamiento / 30% prueba** (holdout de 2.113 clientes). Se entrena en una parte y se evalúa en otra que el modelo **nunca vio**, para tener métricas honestas.
- `stratify=y` garantiza que la proporción de churn (26,5%) sea **igual en train y en test**. Sin esto, por azar el test podría quedar con muy pocos casos de churn y las métricas serían poco fiables.
- `random_state=42` fija la semilla aleatoria → el reparto es **reproducible** (siempre el mismo, en cualquier máquina).

Detalle fino que está bien saber: `entrenar_modelo` primero entrena en el 70% **para medir** en el holdout (métricas honestas que se reportan), y **después re-ajusta el modelo final con el 100% de los datos** etiquetados, para que el modelo que se despliega aproveche toda la información disponible.

f) Las métricas (`_metrics`). Calcula accuracy, precision, **recall**, **F1**, ROC-AUC y **Gini** ($2 * AUC - 1$), más la **matriz de confusión** (TN, FP, FN, TP). Los números del proyecto: recall 79,7%, F1 0,62, Gini 0,69, accuracy 74,2%. En el modo `--eval`, `entrenar_y_evaluar` compara los 4 modelos y reporta **recall en train vs test**: ahí se ve que el RandomForest **sobreajusta** (100% en train, 63,5% en test), razón por la que se descartó a favor de la Regresión Logística.

g) Serialización: `joblib` + `base64` (concepto clave). ¿Cómo se "guarda" un modelo entrenado? Con **serialización**: convertir el objeto Python entrenado en bytes para almacenarlo.

```
joblib.dump(pipe, buf) # modelo entrenado -> bytes
b64 = base64.b64encode(buf.getvalue()).decode("ascii") # bytes -> texto
```

- `joblib` es la librería estándar de sklearn para serializar modelos (más eficiente que `pickle` con arrays de NumPy).
- Como la columna de la base es de **texto**, los bytes se codifican a **base64** (una forma de representar binario como texto). Se guarda en la tabla `modelo_artefacto` junto con sus métricas (en JSONB).
`cargar_modelo_supabase` hace el camino inverso: lee el texto, lo decodifica de base64, y `joblib.load` reconstruye el modelo **idéntico**, listo para predecir **sin re-entrenar**. `guardar_modelo_supabase` hace `TRUNCATE` antes de insertar, así siempre hay **un solo modelo vigente**.

h) `predecir_df`. Aplica el modelo cargado a un DataFrame de clientes y devuelve, por cliente: la clase predicha (`churn_pred`), la **probabilidad** de fuga (`churn_proba` , vía `predict_proba`) y, si hay etiqueta real, si acertó. Esto es lo que llena la tabla `predicciones` que lee el dashboard (2.914 clientes en riesgo).

💡 **Para la defensa:** " `modelo.py` separa entrenar de predecir en tres modos. Usa un `Pipeline` de sklearn que une preprocesamiento y modelo: un `ColumnTransformer` que escala las continuas con `StandardScaler` —porque la regresión logística es sensible a la escala— y aplica `OneHotEncoder` a las categóricas para volverlas números. El desbalance del 26,5% lo tratamos con `class_weight='balanced'`, que sube el recall a 79,7%. Evaluamos con `train_test_split 70/30 stratify=y` para mantener la proporción de churn, y `random_state=42` para reproducibilidad. El modelo entrenado se serializa con `joblib`, se codifica en base64 y se guarda en Supabase, así el predictor lo carga sin re-entrenar."

6. `serve_modelo.py` — La cáscara web: FastAPI y un rol por variable

Qué hace en una frase: envuelve a `modelo.py` en una **API web con FastAPI** y, según una variable de entorno `ROL`, el mismo archivo se comporta como **trainer** (entrena) o como **predictor** (predice). Una sola imagen Docker, dos servicios distintos.

a) El patrón "un rol por variable". Al arrancar lee:

```
ROL = os.getenv("ROL", "predictor").strip().lower()
```

Y según ese valor, **registra unos endpoints u otros**: - `ROL=trainer` → expone `POST /train`: carga los datos de Supabase, llama a `modelo.entrenar_modelo`, y guarda el modelo. Devuelve las métricas del holdout. - `ROL=predictor` (por defecto) → expone `GET /metrics`, `GET /predict/cliente/{id}` y `POST /predict/batch`: carga el modelo desde Supabase y predice **sin entrenar**.

Es el mismo principio que en el pipeline ("misma imagen, distinto comportamiento por variable de entorno"). Los dos servicios **no comparten disco ni código en ejecución**: se comunican **solo a través de Supabase** (la tabla `modelo_artefacto`). El trainer escribe el modelo ahí; el predictor lo lee. Esto es **arquitectura desacoplada de microservicios**.

b) FastAPI. Es el framework que convierte funciones Python en endpoints HTTP. Cada función decorada con `@app.get(...)` o `@app.post(...)` se vuelve una URL que se puede llamar desde el navegador o el dashboard. FastAPI además genera la **documentación Swagger** automáticamente (la demo en vivo de la defensa).

c) El caché del modelo (eficiencia). El predictor no carga el modelo desde la base en cada request: lo guarda en memoria la primera vez (`_cache`) y lo reutiliza:

```
def _modelo():
    if "pipe" not in _cache:
        _cache["pipe"], _cache["met"] = modelo.cargar_modelo_supabase()
    return _cache["pipe"], _cache["met"]
```

Hay un endpoint `POST /reload` que vacía ese caché para forzar recargar el modelo (útil justo después de re-entrenar).

d) Manejo de errores con `HTTPException`. En vez de reventar con un error feo, traduce los problemas a códigos HTTP correctos: si el cliente no existe → **404**; si todavía no hay modelo entrenado → **503** ("servicio no disponible"). Esto hace la API profesional y predecible.

💡 **Para la defensa:** " `serve_modelo.py` expone el modelo con FastAPI. El truco es que la variable de entorno `ROL` decide si el contenedor es trainer (`POST /train`) o predictor (`/metrics` , `/predict/...`): una sola imagen, dos microservicios. No comparten nada en ejecución; se comunican solo por la tabla `modelo_artefacto` de Supabase. El predictor cachea el modelo en memoria para no leerlo de la base en cada request, y traduce los errores a códigos HTTP (404 si el cliente no existe, 503 si aún no hay modelo entrenado)."

Resumen mental (el mapa para no perderse)

Archivo	Etapas	Concepto que el profe puede preguntar
<code>ingesta.py</code>	1. Ingesta	Cascada de fuentes, timestamp para trazabilidad
<code>limpieza.py</code>	2. Limpieza	<code>pd.to_numeric(errors='coerce')</code> , imputación con criterio, <code>.map()</code> , feature derivada
<code>validacion.py</code>	3. Validación	<code>pandera</code> (<code>lazy=True</code>) estructural + reglas semánticas cruzadas
<code>carga_bd.py</code>	4. Carga	SQLAlchemy, <code>lru_cache</code> , SSL, <code>TRUNCATE + INSERT</code> transaccional idempotente
<code>modelo.py</code>	Modelo IA	<code>Pipeline</code> , <code>ColumnTransformer</code> , <code>OneHotEncoder</code> , <code>StandardScaler</code> , <code>class_weight</code> , <code>train_test_split</code> con <code>stratify</code> , <code>joblib</code> +base64
<code>serve_modelo.py</code>	Serving	FastAPI, rol por variable <code>ROL</code> , microservicios desacoplados

Archivos de referencia (rutas absolutas), por si quieren abrirlos durante la defensa: -
C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\ingesta.py -
C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\limpieza.py -
C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\validacion.py -
C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\carga_bd.py -
C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\modelo.py -
C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\serve_modelo.py

Análisis de rendimiento en la nube

Construir un pipeline que *funcione* no es suficiente. En producción también importa **cuánto cuesta** que funcione: cuánto demora, cuántos recursos consume y si se mantiene estable. Esta sección mide eso sobre nuestro sistema real y, sobre todo, identifica **dónde está el cuello de botella** —que es justo lo que la rúbrica pide encontrar.

¿Por qué se mide el rendimiento?

Cuando un pipeline pasa de "corre en mi notebook" a "corre en la nube para un cliente real", aparecen preguntas que antes no importaban. Medimos cuatro cosas:

- **Tiempo de ejecución:** cuánto tarda cada etapa, de principio a fin. Si una predicción tarda 5 minutos, el negocio no la usa.
- **Consumo de CPU:** qué porcentaje del procesador ocupa. Una CPU clavada en 90% sostenido es señal de un proceso mal optimizado o de un servidor a punto de saturarse.
- **Consumo de RAM:** cuánta memoria usa, sobre todo durante el entrenamiento (la etapa más pesada). Si la RAM se dispara, el contenedor puede caerse (un *Out Of Memory*).
- **Estabilidad:** si el pipeline completa todas sus etapas sin errores ni cierres inesperados, y lo hace de forma repetible (no que funcione una vez y falle la siguiente).

La idea de fondo: **el mismo código puede comportarse distinto en local que en la nube**, y hay que tenerlo medido antes de prometerle un nivel de servicio a alguien.

💡 **Para la defensa:** "Medimos rendimiento porque que el pipeline funcione no garantiza que sea usable en producción. Vigilamos cuatro dimensiones: tiempo, CPU, RAM y estabilidad. Lo crítico es que el comportamiento en la nube no es igual al local, y eso hay que tenerlo medido, no supuesto."

¿Cómo lo medimos? (psutil + time.time())

No usamos herramientas exóticas: las mismas dos que recomienda la guía del ramo, en un script propio ([src/benchmark.py](#)).

- `time.time()` mide el tiempo. Es un cronómetro: guardamos la hora antes de una operación (`t0`), la ejecutamos, y restamos la hora de después. La diferencia es cuánto tardó. En el código esto vive en la función `_cron(fn)`, que ejecuta una función y devuelve `(resultado, segundos)`.
- `psutil` (Python System Utilities) mira los recursos del proceso. `proc.memory_info().rss` nos da la RAM real ocupada en memoria física (RSS = *Resident Set Size*), y `psutil.cpu_percent()` el porcentaje de CPU.

Lo importante del diseño del experimento: **leemos el MISMO dataset (7.043 clientes) de dos maneras** y los comparamos. 1. **Lectura local:** desde el CSV validado en disco (`pd.read_csv`). 2. **Lectura en la nube:** desde Supabase (`SELECT * FROM clientes` por una conexión PostgreSQL).

Al ser exactamente los mismos datos, cualquier diferencia de tiempo **no se debe a los datos**, sino al *dónde están* y a *cómo se accede a ellos*. Es una comparación limpia: cambiamos una sola variable (local vs. nube) y dejamos todo lo demás igual. Después medimos también el **entrenamiento** del modelo para ver cuánto pesa el cómputo real.

💡 **Para la defensa:** "Usamos `time.time()` como cronómetro y `psutil` para CPU y RAM. La clave del experimento es que leemos los mismos 7.043 clientes de dos formas —del CSV local y de Supabase— para que la única diferencia sea el origen del dato. Así aislamos el efecto de la nube."

El concepto de CUELLO DE BOTELLA

Un **cuello de botella** es la etapa que más ralentiza el proceso completo —la analogía es literal: en una botella, el líquido sale tan rápido como deje pasar el cuello angosto, da igual lo ancho que sea el resto. En un pipeline, **optimizar cualquier otra etapa no sirve de nada si no atacas la lenta**: el total lo manda el paso más caro.

Por eso medimos etapa por etapa en vez de cronometrar solo el total. El total te dice *qué tan rápido* es; el desglose te dice *dónde* gastar el esfuerzo de mejora. Encontrar el cuello de botella es exactamente el paso que la rúbrica nos pide.

 **Para la defensa:** "El cuello de botella es la etapa que limita la velocidad de todo el pipeline, como el cuello angosto de una botella. Optimizar cualquier otra etapa es perder el tiempo si no atacas esa. Por eso medimos etapa por etapa, no solo el total."

Nuestro resultado

Estos son los números reales de nuestra corrida (`outputs/rendimiento/benchmark.json`):

Operación	Tiempo	Filas
Lectura local (CSV)	0,018 s	7.043
Lectura nube (Supabase)	3,939 s	7.043
Entrenamiento del modelo	1,336 s	7.043
TOTAL	5,293 s	

Recursos: **RAM ≈ 200 MB** (con un incremento de ~82 MB durante la ejecución) y **CPU ≈ 9,9%**.

El hallazgo central salta a la vista:

Leer de la nube tardó 3,94 s; leer el mismo dato local tardó 0,018 s. La nube fue ≈ 219× más lenta.

Y atención al contraste clave: **entrenar el modelo** —el cómputo de verdad, el que "piensa"— **tardó solo 1,34 s, casi tres veces menos que leer los datos de la nube.**

¿Qué significa esto? La latencia de red, no el cómputo

Esto es lo importante de entender y de decir bien:

Nuestro cuello de botella NO es el cómputo. Es la latencia de red.

¿Por qué? La operación matemáticamente más cara —entrenar la regresión logística sobre 7.043 clientes— se resuelve en 1,34 s. En cambio, la operación trivial —traer una tabla que ya existe— se demora 3,94 s. Esos casi 4 segundos **no se gastan calculando nada**: se gastan en el viaje de ida y vuelta de los datos por internet hasta el servidor de Supabase y de regreso. Es **latencia de red**, no esfuerzo de procesador. De hecho la CPU estuvo casi ociosa (~10%): el sistema no estaba trabajando, estaba **esperando** a que llegaran los datos por la red.

Contrastando contra los umbrales sanos que da la propia guía del ramo (para ~10.000 registros: tiempo total 5–15 s, CPU 30–50%, RAM 300–800 MB), nuestro sistema está **cómodamente dentro o por debajo**:

- **Tiempo total 5,3 s** → en el rango aceptable, cerca del piso (rápido). 

- **CPU ~10%** → muy por debajo del 80% de riesgo; sin sobrecarga. ✓
- **RAM ~200 MB** → incluso por debajo del rango, porque 7.043 filas es un dataset chico. ✓
- **Estabilidad** → el pipeline completa las cuatro etapas sin errores. ✓

Y encaja con lo que la guía anticipa: "con **regresión logística**, estos tiempos y consumos son razonables". Elegir un modelo liviano nos da, gratis, un buen perfil de rendimiento.

La mejora propuesta: acercar el cómputo a los datos / cachear

Como sabemos *exactamente* dónde está el problema (la red, no el procesador), la mejora es precisa y barata —no hay que comprar más CPU ni RAM, eso no resolvería nada:

1. **Cachear la lectura:** guardar el resultado de la consulta para no volver a pedir por red lo mismo. Si los 7.043 clientes ya se trajeron una vez, las siguientes corridas leen de una copia local en milisegundos. Pagas la latencia **una vez**, no cada vez.
2. **Acercar el cómputo a los datos:** ejecutar el predictor/entrenador en la **misma región** del servidor de Supabase (ambos en infraestructura cercana). Mientras más corto el cable entre la app y la base, menor la latencia. Hoy el dato viaja más de lo necesario.
3. **Traer solo lo que se usa:** en vez de `SELECT *`, pedir únicamente las columnas necesarias, y para scoring usar lotes (`/predict/batch`) en lugar de mil llamadas individuales —cada llamada paga su propio peaje de red.

La lección de ingeniería: **medir primero, optimizar después**. Sin el benchmark, alguien habría intentado "optimizar el algoritmo" (que ya es rapidísimo, 1,34 s) y no habría tocado el verdadero problema. El número nos dijo dónde mirar.

💡 **Para la defensa:** "Nuestro resultado clave: leer de la nube tardó 3,94 s contra 0,018 s en local, unas 219 veces más. Pero el entrenamiento —el cómputo real— tardó solo 1,34 s. O sea, el cuello de botella NO es el procesador, es la **latencia de red**: la CPU estuvo al 10%, esperando datos, no calculando. El total fueron 5,3 s, dentro de los umbrales sanos. La mejora correcta es **cachear** la lectura y **acercar el cómputo a los datos** (misma región), no comprar más hardware. Y la moraleja: medimos antes de optimizar, porque sin el dato habríamos optimizado la etapa equivocada."

Archivos de respaldo (rutas absolutas): - Datos del benchmark: `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\outputs\rendimiento\benchmark.json` - Script de medición: `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\src\benchmark.py` - Guía oficial de la actividad: `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\EV3\3.2.1 Análisis comparativo de rendimiento del sistema.pdf`

(Nota para quien arma la guía: el archivo 3.2.1 entregado es un **PDF**, no un .txt; lo leí igual. El JSON real reporta `cpu_pct` 9,9% y `overhead_nube_vs_local_x` 218,8, consistente con los ~10% y ~219× usados arriba.)

Auditoría de seguridad y Ley 21.719

Por qué este tema vale el 30% (indicador 9). En la defensa oral no basta con decir "le pusimos seguridad". Tienes que demostrar que entiendes *qué* protegiste, *cómo* lo verificaste y *por qué la ley chilena te obliga a ello*. Esta sección te enseña los **4 frentes de la auditoría** (credenciales, accesos, entorno y logs) y luego conecta cada control técnico con un **principio legal** de la Ley 21.719. Si dominas el mapeo "control técnico ↔ principio legal", tienes el indicador 9 ganado.

¿Qué es una "auditoría de seguridad" y por qué la hicimos?

Una **auditoría de seguridad** es una revisión sistemática que busca responder una pregunta: *¿quién puede tocar nuestros datos, por dónde, y qué pasaría si alguien malintencionado lo intentara?* No es "instalar un antivirus": es revisar el sistema completo (el repositorio en GitHub, la API en Railway y la base en Supabase) buscando puntos débiles, y dejar **evidencia reproducible** de cada revisión.

Organizamos la auditoría en los **4 frentes** que enseña el material del curso (unidad 3.3). La idea de fondo es la **defensa en profundidad**: no confiar en una sola barrera, sino poner varias capas, de modo que si una falla, otra contiene el ataque.

💡 **Para la defensa:** "Nuestra auditoría no fue improvisada: seguimos los 4 frentes del material — credenciales, accesos, entorno y logs— y dejamos evidencia reproducible de cada uno en [outputs/seguridad/auditoria_seguridad.md](#). La filosofía es defensa en profundidad: varias capas, no una sola."

Frente 1 — Credenciales y secretos

Concepto. Un **secreto** es cualquier dato que da acceso al sistema: contraseñas, *tokens* de API, la cadena de conexión a la base (`DATABASE_URL`). La regla de oro es: **los secretos nunca viven en el código** ni en el historial de Git, porque nuestro repositorio es **público** en GitHub (cualquiera en el mundo puede leerlo). Si una contraseña aparece en el código y se sube, ya está comprometida aunque la borres después, porque **queda en el historial de Git** para siempre.

Cómo lo resolvimos (el patrón "secretos solo en el entorno"): - Las credenciales viven únicamente como **variables de entorno** en Railway y Supabase, y en un archivo `.env` **local**. - El código nunca escribe la clave: la **lee** con `os.getenv("DATABASE_URL")`. Es decir, el programa pide el secreto al entorno en tiempo de ejecución, no lo lleva escrito. - El archivo `.env` está **gitignored**: aparece en `.gitignore` (línea 16), así que Git lo ignora y nunca se sube.

Cómo lo verificamos (la evidencia que puedes citar):

Verificación	Cómo	Resultado
¿Hay secretos escritos en el código?	<code>grep -i "password\ token\ secret\ sbp_\ service_role"</code> sobre el repo	Solo placeholders (<code>tu_password</code> , <code>[YOUR-PASSWORD]</code> , <code>'CAMBIAR'</code>) y lecturas <code>os.getenv(...)</code> . 0 secretos reales.
¿ <code>.env</code> está ignorado?	revisar <code>.gitignore</code>	Sí, excluido.
¿ <code>.env</code> se subió alguna vez?	<code>git log --all -- .env</code>	Nunca commiteado.
¿Hubo alguna contraseña en algún commit antiguo?	<code>git log --all -S '<password>'</code>	0 commits.

El `grep` sobre el repo público es la prueba clave: rastreamos todas las palabras que delatarían un secreto y solo encontramos *placeholders* (textos de relleno como `CAMBIAR_EN_PRODUCCION`) y llamadas a `os.getenv`. Cero claves reales.

Además, la conexión a Supabase fuerza `sslmode=require`: el dato viaja **cifrado en tránsito** (TLS), así nadie que intercepte la red ve la contraseña ni los datos.

💡 **Para la defensa:** "Ningún secreto vive en el código. Las credenciales están solo en variables de entorno; el código las lee con `os.getenv`. El `.env` está gitignored y un `grep` sobre el repo público demuestra 0 secretos reales, solo placeholders. Como bonus, la conexión va por TLS con `sslmode=require`." Si te preguntan "¿y si borras la clave del código después?", responde: **"No sirve: queda en el historial de Git. Por eso el secreto nunca debe entrar, no basta con sacarlo."**

Frente 2 — Accesos y permisos (privilegio mínimo)

Este es el frente más técnico y el que más impresiona. Dos conceptos clave:

(a) Privilegio mínimo (*least privilege*). Cada actor del sistema recibe **solo los permisos que estrictamente necesita**, ni uno más. El dashboard solo necesita *leer* datos → entonces solo le damos SELECT, jamás INSERT/UPDATE/DELETE. Así, si alguien roba la credencial del dashboard, **no puede borrar ni alterar nada**, solo leer.

(b) Row-Level Security (RLS) "cerrado por defecto". Supabase expone automáticamente todas las tablas del esquema `public` por una **API REST** (PostgREST). Sin protección, cualquiera con la *anon key* (que es pública por diseño) podría leer las tablas desde Internet. RLS es un mecanismo de PostgreSQL que dice: "*esta tabla no responde a menos que exista una política que lo permita*". Nosotros **activamos RLS en todas las tablas y NO creamos políticas permisivas** → resultado: **cerrado por defecto**. La tabla existe, pero no entrega ni una fila a los roles públicos.

Los tres roles de Supabase que debes saber explicar: - `anon` (anónimo): el rol con el que entra cualquiera que llame a la API REST pública sin loguearse. En nuestro proyecto: `rolcanlogin = false`, `rolbypassrls = false` → **no puede saltarse RLS**, y como hay 0 políticas, **no lee absolutamente nada**. - `authenticated`: el rol de un usuario que sí inició sesión (vía Supabase Auth). Mismo caso: no bypassa RLS y sin políticas no accede. - `postgres` (el rol **dueño**, con el que se conecta nuestro backend): tiene `rolbypassrls = true`, es decir, **ignora RLS** porque es el propietario de las tablas. Por eso la ingesta y la carga del pipeline funcionan. Importante: **es el dueño, NO un superusuario** — distinción que conviene aclarar si te lo preguntan. - (Existe también `service_role`, de uso interno del backend, que también bypassa RLS pero no es de login público.)

Estado verificado (consultando `pg_tables` y `pg_policies` en Supabase):

Tabla	RLS activo	Nº de políticas	Efecto
<code>clientes</code>	Sí	0	cerrado por defecto
<code>predicciones</code>	Sí	0	cerrado por defecto
<code>carga_logs</code>	Sí	0	cerrado por defecto
<code>clientes_rechazados</code>	Sí	0	cerrado por defecto

El propio **advisor de seguridad de Supabase reporta 0 vulnerabilidades**: las 4 notas que aparecen son `INFO: RLS enabled, no policy`, que es **exactamente la postura que buscábamos** (RLS activo, sin puertas abiertas). Una nota informativa, no una alerta.

El rol de solo lectura (`sql/02_rols_seguridad.sql`). Para llevar el privilegio mínimo a producción, definimos un rol `telco_lectura` que: - recibe `GRANT SELECT` solo en `clientes` y `predicciones` (las tablas que el dashboard necesita; **no** en la auditoría interna `carga_logs`), - y al que se le `REVOKE INSERT, UPDATE, DELETE, TRUNCATE` de forma explícita (defensa en profundidad: negamos la escritura por partida doble).

En producción, el dashboard se conectaría como `telco_lectura`, **nunca como el dueño**. Así separamos dos mundos: *escritura* (el pipeline, rol dueño) y *lectura analítica* (el dashboard, rol restringido).

💡 **Para la defensa (este es el punto fuerte del frente):** "La base **no es accesible públicamente**. Activamos Row-Level Security en las 4 tablas sin políticas permisivas: cerrado por defecto. Los roles públicos `anon` y `authenticated` no pueden bypassar RLS y, con 0 políticas, no leen nada. Solo el rol dueño `postgres`, con el que se conecta el backend, bypassa RLS. Y para producción tenemos `telco_lectura`, un rol de **solo SELECT** para el dashboard: aplica privilegio mínimo. El propio advisor de Supabase reporta 0 vulnerabilidades."

💡 **Pregunta trampa frecuente — "¿Por qué RLS con CERO políticas? ¿No es un error?"** Respuesta: **"Al contrario, es intencional. RLS activo + 0 políticas = la tabla niega todo a los roles públicos. El backend funciona porque entra como dueño, que ignora RLS. Si quisiéramos abrir lectura controlada, agregaríamos una política `FOR SELECT` para un rol específico — que es justo lo que hace `telco_lectura`."**

Frente 3 — Entorno de ejecución y dependencias

Concepto. Tu código puede ser perfecto, pero si usa **librerías de terceros con vulnerabilidades conocidas**, heredas esas vulnerabilidades. Una **CVE** (*Common Vulnerabilities and Exposures*) es un identificador público de una vulnerabilidad reportada (ej. [CVE-2024-47874](#)). Auditar el entorno significa: revisar qué versiones de librerías usamos y si alguna tiene CVEs.

La herramienta: `pip-audit`. Escanea `requirements.txt` contra una base de datos de vulnerabilidades conocidas. En nuestro proyecto encontró **10 vulnerabilidades en 3 paquetes**:

Paquete	Versión	CVE	Se corrige en	Criticidad real para nosotros
<code>starlette</code>	0.36.3	CVE-2024-47874 y 6 más	0.40.0 → 1.x	Es transitiva de FastAPI 0.110 (no la usamos directo). Subirla obliga a subir FastAPI → acoplada.
<code>python-dotenv</code>	1.0.0	CVE-2026-28684	1.2.2	Lee el <code>.env</code> . Actualización directa y segura .
<code>pytest</code>	7.4.4	CVE-2025-71176	9.0.3	Solo dependencia de desarrollo/test , no corre en producción.

Lo importante para la defensa no es memorizar las CVEs, sino mostrar **criterio de priorización**: no todas las vulnerabilidades pesan igual. `python-dotenv` se arregla solo (correctivo inmediato), `pytest` ni siquiera corre en producción (riesgo bajo), y `starlette` es **transitiva** (no la instalamos nosotros: viene "arrastrada" por FastAPI), por lo que actualizarla exige planificar una subida conjunta de FastAPI — eso es un cambio **preventivo planificado**, no urgente.

trivy para imágenes/contenedores. Mientras `pip-audit` revisa librerías de Python, `trivy` escanea **imágenes Docker** (el sistema operativo base del contenedor y sus paquetes). Como nuestro despliegue es *cloud-only* (Railway construye la imagen; no tenemos Docker instalado localmente), aplicamos el equivalente gestionado — `pip-audit` + escaneo del repo— y **proponemos** `trivy` para escanear la imagen que Railway construye. Saber nombrar `trivy` y para qué sirve (imágenes, no solo librerías de Python) es un punto que suma.

Exposición de red. Revisamos qué puertos quedan abiertos a Internet: - **Supabase** exige TLS/SSL y autenticación: no acepta conexiones anónimas a la base. El puerto de la BD **no está abierto** a Internet. - **Railway** expone **solo el puerto HTTPS** del servicio FastAPI. Nada de puertos de base de datos colgando en la red.

Es decir: **superficie de ataque mínima** y todo el tráfico **cifrado (SSL/TLS)**.



Para la defensa: "Auditamos las dependencias con `pip-audit`: 10 CVEs en 3 paquetes. El único accionable inmediato es `python-dotenv` (se sube a 1.2.2). `starlette` es transitiva de FastAPI —hay que planificar la subida conjunta— y `pytest` solo corre en test, no en producción. Para las **imágenes Docker** la herramienta es `trivy`, que proponemos sobre la imagen que construye Railway. En red, solo está expuesto el puerto HTTPS de FastAPI; la base no acepta conexiones anónimas y exige TLS." Frase de oro: **"Es el único hallazgo realmente accionable de toda la auditoría, y tenemos plan de mitigación clasificado."**

Frente 4 — Logs de seguridad

Concepto clave: log de *rendimiento* ≠ log de *seguridad*. Esta distinción es la que el material quiere que captes. No es lo mismo monitorear *que el sistema vaya rápido* que monitorear *que nadie esté atacándolo*.

- **Log de auditoría del pipeline (`carga_logs`):** nuestra tabla `carga_logs` registra cada corrida del pipeline — archivo origen, registros leídos/insertados/rechazados, duración y estado (`OK` / `ERROR` / `PARCIAL`)—. Esto da **trazabilidad e integridad del dato**: sabemos exactamente qué entró, cuándo y con qué resultado. Es auditoría operacional.
- **Lectura de seguridad de los logs (Railway/Supabase):** aquí cambia el lente. Un **acceso fallido repetido NO es un error benigno**: muchos intentos fallidos de autenticación seguidos = **posible ataque de fuerza bruta** (alguien probando contraseñas a ciegas). La tarea de seguridad es **clasificar eventos**:
- **Críticos** → accesos denegados, errores de autenticación repetidos (posible intrusión).
- **Advertencias** → latencia alta, reintentos puntuales (problema de rendimiento, no ataque).

💡 **Para la defensa:** "Distinguimos dos tipos de logs. `carga_logs` es auditoría del pipeline: trazabilidad de qué dato entró. Pero la **lectura de seguridad** es distinta: en los logs de Railway/Supabase, **accesos fallidos repetidos = posible fuerza bruta**, no un error cualquiera. Por eso clasificamos eventos críticos (autenticación denegada) frente a advertencias (latencia). La mitigación es revisar periódicamente los accesos y alertar ante picos de fallos."

Síntesis de la auditoría: hallazgo → mitigación

Cada hallazgo se clasifica como **preventivo** (mantener una buena práctica antes de que ocurra el problema) o **correctivo** (arreglar algo ya detectado):

#	Hallazgo	Severidad	Mitigación	Tipo
1	Sin secretos en código/historial	OK	Mantener patrón; revocar tokens tras la evaluación	Preventiva
2	Base cerrada por defecto (RLS)	OK	Aplicar rol de solo lectura al dashboard	Preventiva
3	10 CVEs en dependencias	Media	Actualizar <code>python-dotenv</code> ; planear FastAPI+Starlette	Correctiva + Preventiva
4	Logs no monitoreados por seguridad	Baja	Revisar accesos fallidos periódicamente	Preventiva
5	Free tier se suspende (disponibilidad)	Baja	Despertar servicios antes de demostrar	Preventiva

💡 **Para la defensa:** "El balance es sólido: la base está cerrada y sin secretos expuestos. El único hallazgo de severidad media son las dependencias, con plan de mitigación. Todo lo demás es preventivo: endurecimiento, no remediación de fallas graves."

La Ley: 19.628 modernizada por la Ley 21.719

Qué ley aplica. En Chile la protección de datos personales se rige por la **Ley 19.628**. Esa ley, antigua y débil, fue **reformada y modernizada integralmente por la Ley 21.719**, que **eleva los estándares al nivel europeo del GDPR** (el reglamento europeo, considerado el estándar de oro). Dato clave para citar: la Ley 21.719 entra en **aplicación plena el 01-12-2026**. Como nuestra evaluación es de 2026, es legislación **inminente y vigente**, no hipotética.

Datos personales vs. datos sensibles en el dataset Telco. Esta clasificación es pregunta segura:

- **Datos personales** = información que identifica o se relaciona con una persona. En nuestro dataset:
 - `customerID` → identificador del titular.
 - `gender` , `SeniorCitizen` → atributos de la persona (género, rango etario).
 - `Partner` , `Dependents` → situación familiar.
 - `tenure` , `Contract` , `MonthlyCharges` , `TotalCharges` , `PaymentMethod` → **datos económicos y de la relación contractual**.
- **Datos sensibles** = categorías especiales que la 21.719 protege con más fuerza (salud, biometría, ideología, origen étnico, vida sexual, etc.). **Nuestro dataset NO contiene datos sensibles** en sentido estricto. Pero sí tiene **datos personales económicos y conductuales** que exigen protección.

Saber decir "*no manejamos datos sensibles, pero sí personales de tipo económico-conductual, y por eso igual aplicamos protección*" demuestra que entiendes la diferencia y no exageras ni minimizas.

💡 **Para la defensa:** "Aplica la Ley 19.628 **modernizada por la Ley 21.719**, que sube Chile al nivel del GDPR europeo y entra en vigencia plena el **01-12-2026**. En nuestro dataset hay **datos personales** — `customerID` , género, situación familiar y datos económicos como cargos y método de pago— pero **no datos sensibles** en sentido estricto (salud, biometría). Aun así, los datos económicos exigen protección, y por eso construimos con *compliance by design*."

Compliance by design: mapeo control técnico ↔ principio legal

Este es el **corazón del indicador 9** y lo que distingue una buena defensa de una mediocre. **No defiendas la seguridad por un lado y la ley por otro:** muestra que **cada control técnico cumple un principio legal concreto**. Eso es lo que significa *compliance / privacy by design*: la privacidad no se "añade al final", está **embebida desde el diseño**.

Principio (Ley 21.719 / GDPR)	Qué exige	Control técnico que lo cumple
Finalidad y proporcionalidad	Usar los datos solo para el fin declarado	Los datos se usan solo para predecir churn ; no se recolectan campos extra.
Minimización	Recolectar los datos mínimos necesarios	Usamos <code>customer_id</code> (un identificador, no nombre ni RUT); el modelo usa solo las variables necesarias.
Seguridad y confidencialidad	Proteger los datos contra accesos no autorizados	RLS cerrado por defecto + privilegio mínimo + TLS en tránsito + secretos fuera del código.
Calidad / integridad del dato	Mantener datos exactos y actualizados	Pipeline de limpieza+validación (Eval 2) + auditoría en <code>carga_logs</code> .
Responsabilidad proactiva (<i>accountability</i> , nuevo en la 21.719)	Poder demostrar cumplimiento, no solo cumplir	Controles embebidos desde el diseño; esta misma auditoría es la evidencia documentada.
Derechos ARCO	El titular puede ejercer sus derechos	El pipeline indexa por <code>customer_id</code> y opera full-refresh → permite rectificar o eliminar a un titular.

Los derechos ARCO (memorízalos): **A**cceso (saber qué datos tienen de mí), **R**ectificación (corregir datos erróneos), **C**ancelación (eliminar mis datos), **O**posición (negarme a un tratamiento). Como nuestra base está **indexada por** `customer_id` y el pipeline trabaja en *full-refresh* (reemplaza el dataset completo en cada corrida), podemos **localizar, rectificar o eliminar** los datos de cualquier titular concreto: ahí cumplimos ARCO técnicamente.

Reflexión CIA (la tríada de seguridad de la información, por si la mencionan): - **C**onfidencialidad → RLS + privilegio mínimo + TLS (solo quien debe, ve el dato). - **I**ntegridad → validación + transacciones + auditoría `carga_logs` (el dato no se corrompe). - **D**isponibilidad → servicios gestionados Supabase/Railway. **Riesgo conocido:** el *free tier* se suspende por inactividad → **mitigación:** despertar los servicios antes de operar o demostrar.

💡 **Para la defensa (la frase que cierra el indicador 9):** "No hicimos seguridad y ley por separado: cada control cumple un principio. **RLS y privilegio mínimo = principio de seguridad y confidencialidad. Usar `customer_id` en vez de nombre o RUT = minimización. Usar los datos solo para churn = finalidad. Indexar por `customer_id` con full-refresh = derechos ARCO.** Y lo nuevo de la 21.719, la **responsabilidad proactiva**: tenemos que *demostrar* el cumplimiento, y esta auditoría documentada es justamente esa prueba. Eso es *compliance by design*."

💡 **Si te preguntan "¿qué cambia con la 21.719 respecto a la 19.628?":** "Sube Chile al nivel del GDPR: introduce la **responsabilidad proactiva** (*accountability* — hay que demostrar el cumplimiento, no solo declararlo), refuerza los **derechos ARCO**, define con claridad los **datos sensibles** y crea una autoridad de control. Por eso construimos con privacidad desde el diseño y dejamos evidencia."

Referencias del proyecto (rutas absolutas) usadas en esta sección: - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\outputs\seguridad\auditoria_seguridad.md` — informe completo de la auditoría (4 frentes + mapeo legal + tabla hallazgo→mitigación). - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\sql\01_create_tables.sql` — DDL con activación de **RLS** en las tablas (`ENABLE ROW LEVEL SECURITY`). - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\sql\02_rol_seguridad.sql` — rol de **solo lectura** `telco_lectura` (`GRANT SELECT / REVOKE escritura`), implementación del privilegio mínimo.

Nota: el archivo `3.3.1_Evaluacin_de_seguridad_del_pipeline_y_su_entorno_de_ejecucin.txt` no existe en el proyecto; su contenido (los 4 frentes del material 3.3) ya está incorporado y citado dentro de `auditoria_seguridad.md`, que fue la fuente usada.

Integración BI, el dashboard, y limitaciones/mejoras

¿Qué es Business Intelligence (BI) y por qué importa?

Business Intelligence (Inteligencia de Negocios) es el conjunto de prácticas y herramientas que convierten *datos crudos* en *información comprensible* para tomar decisiones. La idea de fondo es muy simple: **un modelo de IA que nadie entiende no sirve de nada**. Por muy bueno que sea nuestro modelo de churn, si el resultado se queda como un número escondido en una base de datos, el gerente de retención no puede actuar sobre él.

La guía del ramo lo dice textualmente: "*No basta con entrenar un modelo de IA y obtener resultados: es necesario que esos resultados puedan ser interpretados y utilizados por quienes toman decisiones.*" La visualización de datos sirve para tres cosas concretas:

- **Mostrar** los resultados del modelo de forma comprensible (un gráfico se entiende en segundos; una tabla de 2.000 filas, no).
- **Identificar patrones** (por ejemplo: "los clientes con contrato mes a mes se van mucho más").
- **Facilitar la toma de decisiones basada en datos** (a quién llamo primero para retenerlo).

En nuestro caso, la herramienta de BI es un **dashboard hecho en Streamlit**. Streamlit es una librería de Python que convierte un script en una página web interactiva; aparece explícitamente en la lista de herramientas que recomienda el ramo (junto a Power BI, Tableau y Grafana). Lo elegimos porque está en Python —el mismo lenguaje del pipeline y del modelo—, así no tuvimos que aprender otra herramienta ni exportar datos a mano.

💡 **Para la defensa:** "BI es traducir datos en decisiones. Nuestro modelo predice quién se va a ir, pero ese número solo, en una tabla, no le sirve a nadie. El dashboard lo convierte en algo que un encargado de retención puede leer de un vistazo y accionar: a quién llamar primero."

Cómo NUESTRO dashboard integra los resultados (el flujo conectado)

La guía describe el **flujo más utilizado** para conectar todo, y es exactamente el que implementamos:

1. **El pipeline procesa los datos** → deja 7.043 clientes limpios en Supabase.
2. **El modelo genera resultados** → el predictor puntúa a cada cliente y **escribe** las predicciones en la tabla `predicciones` de Supabase.

3. **El dashboard se conecta a esa fuente** → lee de `predicciones` y muestra todo actualizado.

La parte clave —y lo que la rúbrica llama "conexión real con el pipeline"— es que **el dashboard NO recalcula nada ni guarda CSV a mano: lee directo de la base de datos**. En el código (`dashboard/app.py` , función `cargar()`) hay una consulta SQL que cruza tres tablas:

```
SELECT p.customer_id, p.churn_real, p.churn_pred, p.churn_proba, ...  
FROM predicciones p  
LEFT JOIN clientes c ON p.customer_id = c.customer_id
```

Es decir, junta las **predicciones del modelo** (`predicciones`) con los **datos limpios del cliente** (`clientes` , que dejó el pipeline) y con los **logs de la última corrida** (`carga_logs`). Tres salidas de distintas etapas, unidas en un solo panel. Esto es precisamente la *integración* que pide el título de la actividad.

Detalle honesto que conviene saber: las **métricas de calidad** (recall, F1, etc.) no se calculan sobre toda la base, sino que se leen del **registro del modelo** (`modelo_artefacto` , sobre el holdout de 2.113 clientes). El scoring de riesgo, en cambio, se aplica a todos los clientes puntuados. Por eso el panel separa "métricas sobre holdout" de "scoring de la base". Además, si la base está caída (free tier dormido), el dashboard **no se rompe**: cae a un CSV local (`predicciones_test.csv`) como respaldo (el `except` de `cargar()`).

💡 **Para la defensa:** "El dashboard lee la tabla `predicciones` de Supabase con un JOIN: cruza la predicción del modelo con los datos del cliente que dejó el pipeline. No exportamos archivos a mano; si entrenamos de nuevo, el panel se actualiza solo. Eso es lo que la rúbrica llama 'conexión real con el pipeline', no manual."

Los componentes del dashboard (y por qué cada uno)

La guía trae una tabla de "qué visualización usar según el tipo de modelo". Nuestro panel implementa esa tabla casi punto por punto. Lo importante en la defensa es saber decir *qué muestra cada cosa y por qué se eligió ese gráfico*.

1. KPIs (indicadores numéricos). Arriba del todo hay 6 cajas con un número grande cada una: **Recall, F1, Precision, Accuracy, Gini y "Clientes en riesgo"**. Un KPI sirve para mostrar *de un solo vistazo* si el modelo funciona bien. Pusimos el **Recall primero** (79,7%) a propósito, porque es la métrica que prioriza el caso de churn: de los que se van, cuántos detectamos. La caja de Accuracy lleva una advertencia escondida (`help`) que dice: "Engañosa con desbalance: 'todo No-churn' daría ~73,5%" — para no caer en la trampa del accuracy alto.

2. Matriz de confusión (heatmap). Es un mapa de calor 2×2 (`px.imshow`) que muestra los aciertos en la diagonal y los errores fuera de ella. Lo que recalcamos en el texto del panel es el **Falso Negativo**: el cliente que se va y *no* detectamos — "el error más caro". Esto conecta directo con la teoría del ramo: el heatmap "permite ver dónde se equivoca el modelo".

3. Tabla filtrable de errores ("Análisis de errores caso a caso"). Es una tabla dinámica que se puede filtrar por **tipo de caso** (Falso Negativo / Falso Positivo / Acierto), por **contrato** y por un **slider de probabilidad de churn** $\geq$. La guía lo justifica perfecto: "a veces no basta con los promedios, hay que mirar el detalle de cada error". Por defecto el filtro viene marcado solo en los "Falso...", para ir directo a los errores.

4. Tasa de churn por segmento. Un gráfico de barras que agrupa el churn real por **tipo de contrato, servicio de internet, método de pago o antigüedad**. Esto es lo que detecta *patrones de negocio*: por ejemplo, que el contrato mes a mes concentra el abandono. Es la parte más "accionable" para el área comercial.

5. Embudo (funnel) por etapa del pipeline. Abajo hay un embudo ([go.Funnel](#)) con tres niveles: **Ingesta (leídos) → Validados/insertados → Rechazados**. La guía lo llama "Volumen por etapa / flujo del pipeline" y sirve para *detectar cuellos de botella o errores operativos* (cuántos datos entraron, cuántos se descartaron). En nuestro caso muestra cómo de los registros leídos quedaron 7.043 cargados, y cuántos fueron a la tabla de auditoría ([clientes_rechazados](#)).

(También hay una **comparativa de modelos** en barras, que muestra cómo la Regresión Logística balanceada sacrifica accuracy para subir recall frente a las otras.)

💡 **Para la defensa:** "Cada gráfico está elegido a propósito: heatmap para ver los errores del modelo, KPIs para la calidad de un vistazo, tabla filtrable para el detalle caso a caso, barras por segmento para los patrones de negocio, y el embudo para el volumen del pipeline. No es decoración: cada visual responde una pregunta distinta."

Buenas prácticas de visualización aplicadas

El ramo pide aplicar dos principios clave, y los usamos de forma deliberada:

- **Data-ink ratio (Tufte):** mostrar solo la tinta que aporta información y eliminar adornos. En el código se ve: quitamos la barra de color de la matriz ([coloraxis_showscale=False](#)), márgenes mínimos, sin leyendas innecesarias ([showlegend=False](#)), sin títulos de eje redundantes. Cada pixel del panel comunica un dato.
- **Simplicidad visual:** una paleta de **solo tres colores con significado**: azul (neutral/modelo), **rojo = riesgo/error** y **verde = ok/validado**. Nada de gráficos 3D ni efectos. El rojo siempre marca lo malo (falsos negativos, rechazados, churn), así el ojo va solo a lo importante.
- **Diseño iterativo (MVP):** partimos de un panel mínimo y le fuimos agregando paneles según lo que necesitábamos contar.

💡 **Para la defensa:** "Aplicamos data-ink ratio: sacamos todo lo que no aporta —barras de color, leyendas de más— y usamos solo tres colores con significado, donde el rojo siempre es riesgo. Así el panel se lee rápido y sin ruido, que es justo lo que pide la simplicidad visual."

La demo en vivo (vacío → entrenar → predecir → se llena)

Esta es la parte que demuestra que la integración es **real y automática**, no una imagen pegada. El dashboard tiene tres botones que llaman a nuestros microservicios:

1. **Estado inicial: panel vacío.** Con el botón "Vaciar TODO" se truncan las tablas [modelo_artefacto](#) y [predicciones](#). El panel muestra el aviso "*No hay modelo entrenado*" y los KPIs en "—".
2. **Botón 1 — Entrenar:** hace [POST /train](#) al **microservicio trainer**, que entrena la Regresión Logística y guarda el modelo serializado en [modelo_artefacto](#).
3. **Botón 2 — Predecir:** hace [POST /predict/batch](#) al **microservicio predictor**, que carga ese modelo y puntúa a los clientes, escribiendo en [predicciones](#).

4. **El panel se llena solo.** Aquí está lo bonito: un *fragmento con auto-refresh* (`@st.fragment(run_every=3)`), la función `_auto_fill_watcher` revisa la base **cada 3 segundos**; cuando detecta que aparecieron el modelo o las predicciones, recarga el panel completo. Por eso podemos entrenar/predecir **desde el Swagger de la API** (sin tocar el dashboard) y el panel **se actualiza solo**.

Esto demuestra exactamente lo que el ramo pone como ventaja del flujo conectado: "*aseguras que siempre se vean los datos más recientes*" y "*evitas tener que exportar manualmente archivos*".

💡 **Para la defensa:** "En la demo arrancamos con el panel vacío. Entrenamos desde el Swagger del trainer y predecimos desde el del predictor; no tocamos el dashboard. A los pocos segundos el panel se llena solo, porque vigila la base cada 3 segundos. Eso prueba que la integración es automática y en vivo, no una captura."

Limitaciones del sistema (con evidencia medida)

Un punto fino que pide el ramo: **distinguir una limitación de un error**. Un **error** es un fallo del sistema (algo que no funciona, un bug). Una **limitación** es un *límite técnico u operativo conocido y gestionable* del alcance actual: el sistema funciona, pero tiene fronteras. **Ninguna de nuestras limitaciones es un error**. Y cada una se apoya en un número medido, no en una opinión:

Limitación	Evidencia (número real)	Por qué importa
L1 — RandomForest sobreajusta	Recall en train 100% vs test 63,5% (gap ~+0,22)	El RF por defecto <i>memoriza</i> los datos de entrenamiento y no generaliza. Por eso descartamos el RF y elegimos la Regresión Logística.
L2 — Precisión baja / falsas alarmas	LogReg balanceada: recall 79,7% pero precision ~51% (431 falsos positivos)	Señalamos a clientes que en realidad no se iban → la campaña de retención gasta en gente de más.
L3 — Latencia de red domina	Lectura en la nube 3,94 s vs local 0,018 s (~ 219x más lenta)	El cuello de botella no es el cómputo (entrenar toma 1,34 s), es la conexión a Supabase.
L4 — Dataset pequeño y desbalanceado	7.043 filas, solo 26,5% churn	Limita la robustez; con pocos casos de la clase minoritaria el modelo generaliza peor.
L5 — Free tier se duerme	Supabase/Railway se suspenden por inactividad	Riesgo operativo: el primer request tras la inactividad tarda o falla (por eso el dashboard tiene respaldo a CSV).
L6 — Dependencias con CVEs	<code>pip-audit</code> : 10 CVEs en 3 paquetes (starlette/dotenv/pytest)	Superficie de vulnerabilidad conocida; starlette viene acoplada a FastAPI.

💡 **Para la defensa:** "Limitación no es lo mismo que error. El sistema funciona; lo que tiene son fronteras conocidas y medidas. Por ejemplo, que el RandomForest sobreajuste (100% en train, 63,5% en test) no es

un bug: es la razón documentada por la que elegimos la Regresión Logística. Cada limitación la respaldo con un número, no con una corazonada."

Propuestas de mejora (preventiva vs correctiva)

Otra distinción que conviene tener clara para el indicador 10:

- **Mejora correctiva:** *arregla un problema que ya existe y medimos.* (Reacciona a una limitación detectada.)
- **Mejora preventiva:** *evita un problema futuro o refuerza algo que aún no falla.* (Se anticipa.)

Mejora	Atiende	Tipo	Beneficio
M2 — Ajustar el umbral de decisión (no 0,5) según costo FN/FP, o rankear por probabilidad (top-N riesgo)	L2	Correctiva	Sube la precisión sin perder recall → campañas de retención más eficientes. <i>(La más fácil: ya tenemos las probabilidades.)</i>
M1 — Regularizar/tunear el RandomForest (<code>max_depth</code> , <code>min_samples_leaf</code>)	L1	Correctiva	Reduce el sobreajuste; mejor generalización.
M6 — Actualizar dependencias (<code>python-dotenv</code> → 1.2.2, plan upgrade FastAPI+Starlette)	L6	Correctiva + Preventiva	Cierra los 10 CVEs conocidos.
M4 — Validación cruzada k-fold + más feature engineering	L4	Preventiva	Evaluación más robusta y estable (no depende de un solo split).
M3 — Cachear lecturas / materializar vistas (acercar el cómputo a los datos)	L3	Preventiva	Baja la latencia dominante (219×).
M5 — Keep-alive programado o tier pago	L5	Preventiva	Disponibilidad garantizada para la demo y producción.
M7 — Conectar el dashboard con el rol <code>telco_lectura</code> (solo lectura)	seguridad	Preventiva	Privilegio mínimo en producción.

Priorización realista para un equipo de 2 (cómo se implementarían): primero **M2 y M1** (máximo impacto en la calidad del modelo y bajo costo, porque ya tenemos las probabilidades y el pipeline sklearn) → luego **M6 y M7** (seguridad, cambios acotados) → después **M3/M5** (rendimiento/disponibilidad, dependen de la nube). Cada mejora es **incremental** sobre lo construido: ninguna obliga a rehacer el pipeline.

💡 **Para la defensa:** "Correctiva arregla algo que ya medimos mal; preventiva se anticipa a un problema futuro. Nuestra mejora estrella es correctiva y barata: ajustar el umbral de decisión (M2). Como ya guardamos la probabilidad de cada cliente, en vez de cortar en 0,5 podemos rankear por riesgo y atacar el top-N, subiendo la precisión sin sacrificar el recall, sin reentrenar nada."

Rutas de los archivos fuente (todas absolutas): - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\dashboard\app.py` - `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-`

Banco de preguntas y respuestas para la defensa

Por qué esta sección es la más importante. El **70% de la nota es individual y oral**: el docente le pregunta al azar a cualquiera de los dos (Benjamín o Diego) y la respuesta de cada uno define **su propia** calificación. Ese 70% se reparte en tres indicadores: - **Indicador 8 (30%)** — métricas del modelo: justificar las decisiones técnicas y responder preguntas sobre rendimiento. - **Indicador 9 (30%)** — seguridad y ley de datos personales: defender las medidas y vincularlas a la normativa chilena. - **Indicador 10 (10%)** — mejoras: explicar cómo implementarían las optimizaciones propuestas.

Regla de oro: **ambos dominan TODO**. No sirve "Benjamín sabe métricas y Diego seguridad", porque la pregunta de seguridad le puede tocar a Benjamín. Las respuestas de abajo están redactadas para decirse **de memoria, cortas y con el número real**.

Parte A — Las 10 preguntas de la Evaluación Formativa N°3 (resueltas)

La Formativa N°3 es un quiz de selección única de 10 preguntas que el propio docente preparó. **No es la defensa**, pero es la mejor señal de qué conceptos considera clave: si lo preguntó en el quiz, es muy probable que lo pregunte en la oral. Memoricen no solo la letra correcta, sino el **por qué** (que es lo que les pedirán defender en vivo).

P1. ¿Cuál es la diferencia principal entre la variable objetivo y las variables predictoras? Respuesta: D — La variable objetivo es lo que se quiere predecir; las predictoras son las entradas usadas para hacerlo. *Por qué*: en nuestro caso la **variable objetivo es churn** (Sí/No: el cliente se va o se queda) y las **predictoras** son los atributos del cliente (tipo de contrato, antigüedad/tenure, servicio de internet, método de pago, etc.). Las otras opciones son trampas: la objetivo NO tiene por qué ser numérica (la nuestra es categórica binaria), Sí depende de datos históricos (aprende del pasado), y las predictoras Sí influyen en el resultado.

P2. En los logs, ¿qué significan accesos fallidos repetidos? Respuesta: A — Posibles intentos de fuerza bruta / hackeo. *Por qué*: muchos intentos de login fallidos seguidos son la firma típica de un **ataque de fuerza bruta** (probar contraseñas en masa). Por eso se monitorean como **evento de seguridad**, no como un problema de rendimiento ni un error benigno.

P3. ¿Cuál de estas acciones pertenece a la evaluación (no al entrenamiento)? Respuesta: D — Comparar predicciones con valores reales y calcular las métricas definidas. *Por qué*: es exactamente la separación **trainer vs predictor** de nuestro sistema. **Entrenar** = ajustar el modelo a los datos (`fit()`, cargar X_train, tocar hiperparámetros). **Evaluar** = ya con el modelo entrenado, predecir sobre el holdout y comparar contra la verdad para sacar recall, F1, Gini. Las opciones A, B y C son todas parte del entrenamiento.

P4. ¿Qué acción concreta aplica el principio de privilegios mínimos? Respuesta: B — Definir roles de acceso mínimo (IAM, roles de BD). *Por qué*: privilegio mínimo = **cada quien solo lo justo para su tarea**. En el proyecto

lo aplicamos con un **rol solo-lectura** para el dashboard/analítica y el rol dueño solo para el pipeline. Usar una cuenta admin para todo, guardar claves en el código o abrir más puertos son lo **contrario**.

P5. ¿Cuál de las siguientes es una métrica de regresión (no de clasificación)? Respuesta: D — MAE (Error Absoluto Medio). *Por qué:* F1, recall y precision son métricas de **clasificación** (predecir una clase, como churn Sí/No) — las que usamos nosotros. El MAE mide error de un número continuo, propio de **regresión**. Esto refuerza que nuestro problema es **clasificación binaria**, por eso reportamos recall y matriz de confusión, no MAE.

P6. ¿Por qué es importante la estratificación al separar entrenamiento y prueba en clasificación? Respuesta: D — Para que ambas clases queden representadas de forma equilibrada en ambos conjuntos. *Por qué:* con un **desbalance de 26,5%** (solo 1 de cada 4 clientes se va), un split al azar podría dejar pocos casos de churn en el test. Estratificar **mantiene ese 26,5% tanto en train como en test**, de modo que las métricas sean confiables. No tiene nada que ver con velocidad ni memoria.

P7. ¿Cuál es el resultado esperado al evaluar configuraciones inseguras o accesos no protegidos? Respuesta: B — Que los accesos estén controlados y limitados a quienes corresponda. *Por qué:* el objetivo de una auditoría de seguridad es **cerrar lo abierto**: por eso nuestra base tiene **RLS cerrado por defecto** (Row-Level Security) y solo abrimos lo justo. Dejar la base pública, abrir todos los puertos o hacer la autenticación opcional es justo lo que la auditoría busca **eliminar**.

P8. ¿Cuál de estos elementos NO forma parte de la revisión de seguridad inicial del pipeline? Respuesta: A — Colores del dashboard. *Por qué:* la revisión de seguridad mira **puertos abiertos, contraseñas/claves y accesos/permisos**. Los colores del dashboard son estética, no seguridad. (Ojo, es pregunta "trampa por negación": piden lo que **NO** corresponde.)

P9. ¿Qué herramienta ayuda a detectar vulnerabilidades en imágenes/librerías del entorno? Respuesta: D — trivy. *Por qué:* **trivy** es un escáner de vulnerabilidades de imágenes de contenedor y dependencias (en la misma familia que **pip-audit**, que nosotros usamos y que encontró **10 CVEs**). **netstat** mira conexiones de red, **ping** prueba conectividad y **tail -f** solo sigue logs: ninguno escanea vulnerabilidades.

P10. ¿Qué describe mejor el sobreajuste (overfitting)? Respuesta: C — Alto desempeño en entrenamiento, pero bajo en prueba (memoriza en vez de generalizar). *Por qué:* es **literalmente lo que nos pasó con el RandomForest**: recall **100% en train** y solo **63,5% en test**. Memorizó los datos de entrenamiento y no generaliza. La opción B (malo en ambos) es *underfitting*, no overfitting.

💡 **Para la defensa:** estas 10 preguntas son el "mapa mental" del profe — fíjense que **6 de 10 son de seguridad/ley y métricas**, justo el 60% individual. Si dominan el *por qué* de cada una (no solo la letra), ya tienen cubierto el grueso de las preguntas orales.

Parte B — Banco de preguntas probables de la defensa (respuestas modelo para decir en voz alta)

Agrupadas por indicador. Cada respuesta está pensada para decirse en 15–25 segundos, con el número real adentro.

Indicador 8 — Métricas del modelo (30%)

P. ¿Por qué priorizaron el recall y no el accuracy? R. Porque con un desbalance del 26,5%, el accuracy engaña: un modelo que dijera "nadie se va" acertaría un 73,5% sin detectar a un solo cliente en riesgo. El recall mide cuántos de los que de verdad se van logramos detectar; nuestro modelo alcanza **79,7% (detecta 447 de 561 fugas)**. En retención, no detectar una fuga es el error más caro, así que optimizamos recall.

P. ¿Qué es un Falso Negativo (FN) en este caso y por qué es el error más grave? R. Un FN es un cliente que **sí se va a ir, pero el modelo dijo que no**. Es el peor error porque lo perdemos sin haber hecho nada: no le ofrecimos retención. Un Falso Positivo (predecir que se va y se queda) solo cuesta una llamada o un descuento innecesario; el FN cuesta el cliente entero. Por eso bajar los FN (subir recall) es la prioridad.

P. ¿Qué es la matriz de confusión y qué muestra la de ustedes? R. Es la tabla que cruza lo que el modelo predijo contra la realidad: Verdaderos Positivos, Verdaderos Negativos, Falsos Positivos y Falsos Negativos. Sobre el holdout de **2.113 clientes**, nos interesan sobre todo los **447 churn detectados (TP)** frente a los FN. De ahí salen recall, precisión y F1; es la base de toda la interpretación.

P. ¿Qué es el coeficiente de Gini y qué significa el 0,69 de ustedes? R. El Gini mide el **poder discriminante** del modelo, qué tan bien separa a los que se van de los que se quedan. Se calcula como **Gini = 2 × AUC – 1**; con nuestra ROC-AUC de 0,85 da **0,69**. La escala va de 0 (azar, una moneda) a 1 (perfecto), así que 0,69 es un modelo **bueno**, claramente lejos del azar.

P. ¿Qué es el overfitting y dónde lo vieron en su proyecto? R. Overfitting es cuando el modelo **memoriza** el entrenamiento en vez de aprender el patrón, y por eso falla con datos nuevos. Lo vimos en el **RandomForest por defecto: 100% de recall en train y solo 63,5% en test** — esa brecha enorme es la señal. Por eso lo descartamos y elegimos la **Regresión Logística balanceada**, que generaliza mejor (recall 79,7% en test).

P. ¿Por qué eligieron Regresión Logística balanceada en vez del RandomForest? R. Por tres razones: el RF **sobreajustaba** (100% train / 63,5% test), la LogReg balanceada da mejor recall en datos nuevos (**79,7%**), y además es **interpretable** — podemos explicar por qué clasifica a un cliente como riesgo, lo que importa para retención y para la ley de datos. El "balanceada" viene del `class_weight`, que compensa el desbalance del 26,5%.

P. ¿Por qué estratificaron el split 70/30? R. Para que la proporción de churn (26,5%) se mantenga igual en train y en test. Con desbalance, un split aleatorio podría dejar muy pocos casos de fuga en el test y dar métricas poco confiables. El 70/30 estratificado nos deja **2.113 clientes en el holdout** con la misma proporción de clases que el total.

P. ¿Por qué `class_weight` y no SMOTE para el desbalance? R. Porque el desbalance es **moderado** (26,5%, no 1%). `class_weight='balanced'` resuelve el problema en **una línea**, haciendo que el modelo "pese" más los errores sobre la clase minoritaria, **sin inventar datos sintéticos** como SMOTE y sin riesgo de fuga de datos. Es la solución más simple y robusta para nuestro caso.

P. ¿Qué diferencia hay entre entrenar (train) y predecir (predict) en su arquitectura? R. Las separamos en **dos microservicios**. El **trainer** (`POST /train`) ajusta el modelo con los datos y guarda el modelo serializado en la tabla `modelo_artefacto` de Supabase. El **predictor** (`/metrics`, `/predict/cliente/{id}`, `/predict/batch`) **carga ese modelo ya entrenado y solo predice, sin re-entrenar**. Es la misma imagen Docker; el rol lo decide la variable `ROL` — patrón "un contenedor por capa".

P. ¿Por qué accuracy 74,2% no es una buena noticia por sí sola? R. Porque el "modelo tonto" que dice que nadie se va ya da **73,5%** solo por el desbalance. Nuestro 74,2% apenas lo supera en accuracy, pero la diferencia

real está en que **sí detectamos fugas** (recall 79,7%). Por eso nunca defendemos el modelo con el accuracy solo; lo acompañamos siempre de recall, F1 (0,62) y Gini (0,69).

Indicador 9 — Seguridad y ley de datos (30%)

P. ¿Qué ley de protección de datos aplica a este proyecto? R. En Chile aplica la **Ley 19.628 de datos personales, modernizada por la Ley 21.719**, que la lleva a un **nivel tipo GDPR** y entra en vigencia plena el **01-12-2026**. Como esa fecha es inminente, diseñamos con enfoque **"compliance by design"**: incorporamos las medidas desde el inicio en vez de parchar después.

P. ¿Qué datos personales maneja el sistema? ¿Hay datos sensibles? R. Maneja datos **personales** del cliente: género, si es adulto mayor, situación familiar (pareja/dependientes) y datos económicos como cargos y método de pago. **No hay datos sensibles en sentido estricto** (la ley reserva "sensibles" para salud, origen étnico, religión, ideología, vida sexual, datos biométricos). Aun así los tratamos con cuidado porque siguen siendo personales y protegidos.

P. ¿Cómo aplican el principio de privilegio mínimo? R. Cada componente recibe **solo los permisos que necesita**. El pipeline de ingesta usa un rol con permisos de escritura; el dashboard y la analítica usan un **rol solo-lectura** que no puede modificar nada; y para los roles públicos la base está **cerrada por defecto** con RLS. Así, si una credencial se filtra, el daño posible es mínimo.

P. ¿Qué es RLS y cómo lo usan? R. RLS es **Row-Level Security**, una función de PostgreSQL/Supabase que controla el acceso **fila por fila**. Lo dejamos **cerrado por defecto**: si una política no autoriza explícitamente, no se ve ningún dato. Es la diferencia entre "abierto salvo que lo cierres" y "cerrado salvo que lo abras" — nosotros elegimos lo segundo, que es lo seguro.

P. ¿Qué es trivy / pip-audit y qué encontraron? R. Son **escáneres de vulnerabilidades**: trivy revisa imágenes de contenedor y librerías, y pip-audit revisa las dependencias de Python contra bases públicas de CVEs. Nosotros corrimos **pip-audit y encontró 10 CVEs**, con su plan de actualización (por ejemplo, subir [python-dotenv](#)). Es parte de revisar la cadena de suministro de software, no solo nuestro código.

P. ¿Qué significan los accesos fallidos repetidos en los logs? R. Son la señal típica de un **ataque de fuerza bruta**: alguien probando contraseñas en masa. Por eso los logs no son solo para depurar — los tratamos como **eventos de seguridad** que hay que monitorear para detectar intentos de intrusión y responder a tiempo.

P. ¿Cómo evitan que se filtren secretos (claves, contraseñas) en el repositorio? R. Las credenciales nunca van en el código: viven en **variables de entorno**, y el [.env](#) está en [.gitignore](#). Lo verificamos con **grep sobre el código y sobre el historial de git** para confirmar **cero filtraciones**. Ese fue uno de los cuatro frentes de la auditoría, junto con RLS, escaneo de dependencias y logs.

P. ¿Cuáles fueron los cuatro frentes de su auditoría de seguridad? R. Uno, **cero secretos** en el repo (verificado con grep + historial git). Dos, **RLS cerrado por defecto** con rol solo-lectura (privilegio mínimo). Tres, **escaneo de dependencias** con pip-audit (10 CVEs y su plan). Cuatro, **monitoreo de logs** buscando accesos fallidos repetidos. Todo bajo el paraguas "compliance by design" hacia la Ley 21.719.

P. Si la base es solo-lectura para el dashboard, ¿cómo se llena con predicciones? R. Porque el rol que **escribe** predicciones es el del microservicio predictor, no el del dashboard. El dashboard solo **lee** [modelo_artefacto](#) y la tabla de predicciones para mostrarlas. Es exactamente privilegio mínimo: el que muestra datos no necesita permiso para modificarlos, y no lo tiene.

Indicador 10 — Limitaciones y mejoras (10%)

P. ¿Qué limitaciones reconocen en el sistema? R. Tres principales: la **precisión es baja** (priorizamos recall, así que generamos varios falsos positivos), el RandomForest **sobreajustaba** (por eso lo descartamos), y el **cuello de botella es la latencia de red**, no el cómputo — el benchmark mostró **3,9 s en la nube vs 0,018 s en local**. Reconocerlas es parte de la nota; lo importante es que cada una tiene una mejora asociada.

P. ¿Qué mejoras propondrían y cómo las implementarían? R. Tres concretas. Una, **ajustar el umbral de decisión** (en vez del 0,5 por defecto) para balancear mejor recall y precisión según el costo real de retención. Dos, **regularizar el RandomForest** (limitar `max_depth`, podar) para reducir su overfitting si lo quisiéramos usar. Tres, **actualizar dependencias** para cerrar los 10 CVEs de pip-audit (p. ej. `python-dotenv`). Y para la latencia, cachear o acercar el cómputo a la base.

P. El cuello de botella es la red, no el modelo. ¿Qué harían al respecto? R. Como el benchmark mostró que los 3,9 s son **latencia de red**, no cómputo (local: 0,018 s), atacar el modelo no ayudaría. Las mejoras correctas son de **arquitectura**: cachear predicciones frecuentes, hacer scoring por lotes en vez de cliente por cliente, o acercar el predictor a la base de datos para reducir los viajes de red.

💡 **Para la defensa:** cuando les pregunten una mejora, no digan solo *qué* harían — digan **cómo** y **por qué** (el indicador 10 exige "plan claro y realista para implementar"). Plantilla de respuesta: *"El problema X lo arreglaría con la técnica Y, porque produce el beneficio Z"*. Ejemplo: *"La precisión baja la subiría ajustando el umbral de decisión, porque permite mover el balance recall/precisión según cuánto cuesta una retención, sin re-entrenar el modelo."*

💡 **Para la defensa (cierre de toda la sección):** tres reglas para la oral. (1) **Siempre con el número real** — no digan "buen recall", digan **"recall 79,7%, detecta 447 de 561"**. (2) **No leer las slides** — la rúbrica penaliza leer; el apoyo visual complementa, no se recita. (3) **Conecten dato con decisión** — toda métrica termina en una acción de negocio: "detectamos al cliente en riesgo → el equipo de retención lo llama antes de que se vaya". Eso es lo que separa el 100% ("justifica con claridad y precisión") del 60% ("explicaciones confusas o incompletas").

Archivos fuente leídos: `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\EV3\EV FORMATIVA.pdf` (las 10 preguntas del quiz), `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\EV3\EV PARCIAL 3_ITY1101_ESTUDIANTE.pdf` (rúbrica: indicadores 8=30%, 9=30%, 10=10%), `C:\Users\USER\Desktop\PROYECTO GESTION DATOS IA\telco-churn-pipeline\outputs\guion_defensa.md` (preguntas anticipadas y números del proyecto).